

University of Clermont Auvergne

ISIMA



Master Research Thesis

Multi-Agent Reinforcement Learning for 3D Urban Environment Mapping under Energy and Communication Constraints

Author: Prince Gedeon GUEDJE

supervisor: Viet Hung NGUYEN

*A research proposal submitted in partial fulfillment of the requirements for the Master 2
International in Computer Science
ISIMA (Higher Institute of Computer Modeling and Applications)*

March 2, 2026

“I am the master of my fate, I am the captain of my soul”
from Invictus, by William Ernest Henley

Abstract

Many regions around the world still lack reliable addressing and geospatial systems, limiting the effectiveness of urban planning, logistics, and emergency response. Traditional mapping methods are slow and expensive. Drone swarms offer a scalable alternative, but coordinating multiple drones safely in 3D environments is very difficult.

In this thesis, we study the use of **Multi-Agent Reinforcement Learning** to solve this problem for autonomous urban mapping. We model the system as a **Constrained Decentralized Partially Observable Markov Decision Process**. This means each drone makes its own decisions based on local observations, without needing a central computer to control everything. The drones share information with nearby peers using graph-based communication.

The system rewards the drones for exploring and mapping new areas. At the same time, we use a hybrid approach to handle safety and operational limits. We use Lagrangian relaxation to manage long-term resources, such as battery life. For immediate safety, we use Control Barrier Functions (CBFs) to prevent collisions and keep drones out of restricted no-fly zones. Finally, we test our method in a realistic simulation. The simulation uses accurate quadcopter dynamics and LiDAR sensors. Using collaborative multi-robot SLAM, the drones combine their individual data to build one complete, shared global map.

Keywords: Multi-Agent Reinforcement Learning, Constrained Optimization, Decentralized Control, Graph Neural Networks, UAV Swarms

Contents

List of Figures	iv
List of Tables	v
Glossary	vi
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	2
1.3 Research questions	2
1.4 Proposed Approach and Contributions	2
1.5 Structure of the document	3
2 Background	4
2.1 Fundamentals of UAV Systems	4
2.1.1 UAV Dynamics and Control	4
2.1.2 Sensing and Perception	5
2.1.3 Environment Representation	5
2.1.3.1 The SLAM Problem	5
2.1.3.2 3D Environment Representations	6
2.2 Reinforcement Learning and Multi-Agent Decision-Making	7
2.2.1 Single-Agent Reinforcement Learning (RL)	7
2.2.2 Multi-Agent Reinforcement Learning (MARL)	10
2.2.3 Formal Models for Cooperative MARL	12
3 Literature review	15
3.1 Classical Multi-UAV Coverage and Exploration	15
3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication	16
3.3 Reward Design, Constraints, and Safe Reinforcement Learning	22

3.4	A Review of Simulation Platforms for Reinforcement Learning in UAV Applications	25
4	Proposed Methodology and System Architecture	27
4.1	Proposed Approach	27
4.2	Formal Model and Constrained Problem Formulation	28
4.2.1	Operational Constraints	29
4.2.2	Constrained Optimization Objective	30
4.3	Learning Architecture	31
4.3.1	Algorithmic Selection	31
4.3.2	Graph-Based Communication and Swarm Coordination	33
4.3.3	Hybrid Constraint Management Architecture	34
4.3.3.1	Lagrangian Relaxation for Cumulative Constraints	34
4.3.3.2	Control Barrier Functions for Hard Safety	34
4.3.3.3	Analytical Safety	35
4.3.4	Reward Function Design and Multi-Agent Credit Assignment	36
5	Implementation and Experimental Setup	38
5.1	Simulation Environment	38
5.1.1	Physics Engine and Procedural World	38
5.1.2	UAV Swarm and Hardware Modeling	39
5.2	Software Architecture	39
5.3	Training Pipeline	41
5.3.1	Neural Network Architecture and Hyperparameters	41
5.3.2	Role of Multi-Robot Graph SLAM within the Reinforcement Learning Loop	43
5.4	Experimental Design and Monitoring	44
6	Results and Discussion	46
6.1	Experimental Results and Proposed Solutions	46
6.1.1	Validation and Execution Metrics	46
6.1.2	Action Space Complexity and Deployment Strategy	47
6.2	Implementation Challenges	48
6.3	Discussion and Critical Analysis	48
6.4	Perspectives and Future Work	49
7	Conclusion	50

CONTENTS

References	51
------------	----

List of Figures

2.1	A non-exhaustive, but useful taxonomy of algorithms in modern RL. (Ex- tracted from (1))	8
2.2	Schematic of multi-agent reinforcement learning. . (Extracted from (2)) . .	11
2.3	Hierarchy of game models . (Extracted from (2))	13
3.1	Comparison of Training and Execution Paradigms in MARL	17
3.2	Expanded comparison of MARL learning techniques (Extracted from (3)) .	18
3.3	General architecture of Swarm-SLAM showing local pose-graph estimation, selective loop-closure exchange, and distributed map optimisation. (Ex- tracted from (4))	22
3.4	Comparative review of widely used UAV simulators.(extracted from (5)) . .	25
4.1	Proposed decentralized multi-UAV exploration and mapping architecture.	28
5.1	Simulation procedurally generated 3D urban canyon in Gazebo Harmonic .	38
5.2	Simulated X-frame multi-rotor UAV with onboard sensors.	39
5.3	Modular multi-layer architecture of the proposed decentralized MARL frame- work. The system separates deployment, orchestration, reinforcement learn- ing, and simulation modules, interconnected through a ROS 2 communica- tion layer.	40
6.1	Procedurally generated urban environment in Gazebo Harmonic featuring randomized building geometries and No-Fly Zones	46
6.2	Real-time dual visualization proving the functional link between the physi- cal Gazebo simulation on the right and the MARL perception state on the left.	47

List of Tables

4.1	Fixed Reward Weights for Swarm Convergence	37
5.1	Complete mapping of Gazebo and ROS 2 topics via the <code>ros_gz_bridge</code> . . .	41
5.2	Core Hyperparameters for the MAPPO Actor-Critic Training Pipeline. . . .	43

Glossary

CBF	Control Barrier Function; a formal safety mechanism that ensures system states remain within a safe set by constraining control inputs via quadratic programming.
CPP	Coverage Path Planning; the task of planning paths to ensure complete exploration of an area.
Dec-POMDP	Decentralized Partially Observable Markov Decision Process; a formal model for multi-agent decision making under uncertainty and partial observability.
GNN	Graph Neural Network; a neural network architecture designed to process data represented as graphs, useful for modeling agent communication and relationships.
Hierarchical Reward	A reward function structured in multiple levels to guide agents toward emergent behaviors while addressing MARL challenges

such as credit assignment and non-stationarity.

MADDPG	Multi-Agent Deep Deterministic Policy Gradient; a MARL algorithm for continuous action spaces using actor-critic models with centralized training.
MAPPO	Multi-Agent Proximal Policy Optimization; a MARL algorithm for decentralized multi-agent policy optimization with a centralized critic.
MARL	Multi-Agent Reinforcement Learning; a framework where multiple agents learn to make decisions through trial-and-error interactions in a shared environment.
QMIX	A MARL algorithm using a centralized mixing network for learning joint action-value functions while allowing decentralized execution.
RL	Reinforcement Learning; a machine learning paradigm where an agent learns to maximize cumulative rewards by interacting with an environment.
SLAM	Simultaneous Localization and Mapping; a method used by agents to build a map of an unknown environment while tracking their own location.
UAV	Unmanned Aerial Vehicle; an autonomous agent capable of flying and performing sensing tasks in 3D environments.

1

Introduction

1.1 Context

Reliable acquisition and continuous updating of urban geospatial data are essential for urban planning, infrastructure management, logistics, and emergency response. In many regions, however, national address systems and building inventories remain incomplete or entirely absent, forcing countries to rely on manual field surveys and isolated aerial campaigns. These approaches are slow, costly, and resource-intensive. Furthermore, they contribute to significant carbon emissions, particularly when dependent on manned aviation and repeated ground operations. As urban areas become denser and undergo rapid transformation, conventional data collection methods are unable to provide timely, high-resolution information at scale.⁽⁶⁾

Small Unmanned Aerial Vehicles (UAVs) have recently emerged as promising alternatives for urban data acquisition. These platforms provide flexible, on-demand, high-resolution sensing at reduced operational costs and lower environmental impact.⁽⁷⁾ Advances in computer vision and deep learning facilitate the automatic extraction of key urban features, such as building footprints, façades, and road structures, which directly support address systems and geographic information infrastructures. Concurrently, open-source digital twins and high-fidelity physics-based simulators offer realistic environments for the design, testing, and validation of autonomous UAV strategies prior to deployment. These developments reduce operational risks and accelerate innovation.

Despite significant progress in technology, achieving fully autonomous and scalable urban data acquisition remains challenging. Urban environments are structured and three-dimensional. They have tall buildings, narrow streets, occlusions, and complex visibility. Additionally, UAVs must comply with strict regulations, including restricted flight zones,

altitude limits, and safety protocols. At the same time, they must manage limited energy and intermittent communication. Moreover, Coverage Path Planning (CPP) and Multi-Robot Coverage Path Planning (MCP) become especially complex. This is true even in simple two-dimensional scenarios, where these tasks are NP-hard and typically require heuristic or approximate methods(8). When multi-UAV systems operate in real-world three-dimensional urban environments, operational constraints and partial observability introduce additional challenges.

Consequently, the disparity between existing techniques and the requirements for autonomous, resilient, and efficient data acquisition is increasingly evident. Addressing this gap serves as the main motivation for the research presented in this thesis.

1.2 Problem Statement

The main objective of this thesis is to design a fully autonomous and resilient multi-UAV system that efficiently provides 3D coverage in complex urban environments while operating under strict constraints, such as limited energy, regulatory restrictions, intermittent communication, and partial observability.

1.3 Research questions

To address this problem, the thesis examines the following sub-questions:

1. How can the multi-UAV coverage problem be formulated as a decentralized decision process suitable for complex 3D urban environments?
2. How can the reward structure be designed to balance coverage quality, energy use, resilience, and regulatory compliance?
3. How can a high-fidelity digital twin and an end-to-end acquisition-to-mapping pipeline be built to support the training and evaluation of autonomous multi-UAV strategies?

1.4 Proposed Approach and Contributions

Imagine a fleet of unmanned aerial vehicles (UAVs) making rapid decisions as they explore a complex urban environment. These UAVs exchange limited information with nearby agents when possible and return to the base for recharging as needed. Each agent generates a detailed three-dimensional map from real-time sensor data. These maps are integrated

at a central base to create a comprehensive representation of the area. The goal is to develop an exploration strategy that is efficient, energy-aware, and robust against the unpredictable challenges of three-dimensional urban environments.

Contributions

1. This study proposes a decentralized, learning-based exploration strategy that incorporates constraints on energy, safety, communication, and three-dimensional data acquisition within a unified, adaptive autonomous decision-making framework.
2. Additionally, a method is introduced for fusing local point cloud observations, yielding a coherent global map suitable for urban-scale analysis.
3. Finally, a simulation framework is developed to facilitate the exploration and comparison of scenarios with realistic UAV dynamics, supporting the visualization and evaluation of various strategies.

1.5 Structure of the document

The document is divided into three chapters, namely:

- **Background.** We will start by presenting the theoretical foundations necessary for our work, including UAV, Reinforcement Learning, multi-agent systems, coverage path planning, and key concepts in 3D mapping.
- **Related Work.** This chapter reviews the state-of-the-art, discussing classical approaches, MARL, reward shaping, communication strategies, fault tolerance, and challenges in autonomous multi-agent exploration.
- **Proposed Methodology and System Architecture.** This chapter details the proposed framework and methodology, including hierarchical reward design, agent modeling, graph-based communication, simulation environment, and the plan to evaluate MARL algorithms under operational constraints.

2

Background

This chapter defines the key concepts, theories, and tools used in this thesis. It provides the toolbox necessary to understand the methodology.

2.1 Fundamentals of UAV Systems

An Unmanned Aerial Vehicle (UAV), commonly referred to as a drone, is an aircraft that operates without an onboard pilot and can be controlled remotely or function autonomously. This study focuses on quadcopters, whose ability to hover stably and maneuver accurately makes them well-suited for operation in complex environments.

2.1.1 UAV Dynamics and Control

A quadcopter (quadrotor) is a rigid-body aerial platform whose movement in a three-dimensional space is defined by a six-degree-of-freedom (6DOF) pose. The state vector $\mathbf{x}$ consists of:

- the position $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ of the center of mass of the UAV's expressed in a global reference frame,
- the orientation $R \in SO(3)$, a rotation matrix describing the attitude (roll, pitch, yaw) that maps the body frame to the global frame.

The system dynamics establish a relationship between the platform's state and its derivatives, specifically the linear velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$, as well as the control inputs U_t , which correspond to the thrust generated by the four motors(9). The primary control

objective is to determine appropriate U_t values that guide the state $\mathbf{x}_t$ along a specified trajectory $\mathbf{x}_{\text{ref}}(t)$ while adhering to the platform's physical constraints. This process depends on obtaining an accurate estimate of the current state $\mathbf{x}_t$ from the perception subsystem.

2.1.2 Sensing and Perception

Sensing refers to the physical hardware that collects raw data from the world, while perception is a computational process that interprets this data to infer the UAV's state (pose) and the three-dimensional structure of the environment.

The primary sensors used in autonomous navigation systems generate the following types of raw data:

- **Inertial Measurement Unit (IMU):** Provides measurements of the UAV's linear acceleration $\mathbf{a}$ and angular velocity $\boldsymbol{\omega}$, which are essential for estimating its motion and orientation.
- **Camera:** Captures visual information in the form of images $I(u, v)$, where (u, v) denote pixel coordinates.
- **LiDAR:** An active sensing device that measures distances to surrounding objects by emitting laser pulses. It returns a three-dimensional point cloud $\mathbf{q}_i \in \mathbb{R}^3$, providing precise geometric information about the environment.

Once raw data is collected, the perception subsystem processes it using algorithms such as visual odometry, simultaneous localization and mapping (SLAM), and 3D reconstruction. These methods interpret sensor signals to estimate the UAV's 6DOF pose $\mathbf{x}_t$ and to generate a **map of the environment**.

2.1.3 Environment Representation

2.1.3.1 The SLAM Problem

Simultaneous Localization and Mapping (10) is a foundational problem in robotics and autonomous navigation. It helps an autonomous vehicle estimate its own position while it builds a map of an unknown environment. SLAM is especially important for Unmanned Aerial Vehicles (UAVs) that operate in areas where GPS is unavailable or unreliable.

Theoretical Framework and Probabilistic Modeling

At its core, SLAM is a probabilistic state estimation problem, where the goal is to compute the posterior distribution of the UAV's trajectory $\mathbf{X} = \mathbf{x}_1, \dots, \mathbf{x}_t$ and the map m given a sequence of observations.

$$P(\mathbf{X}, m \mid \mathbf{Z}, \mathbf{U})$$

Where the terms represent:

- $P(\dots)$ The probability distribution
- $\mathbf{X}$ The **trajectory** (the set of all robot poses $\mathbf{x}_t$)
- m The **map** of the environment
- $\mathbf{Z}$ The set of all sensor **observations** (what the robot sees)
- $\mathbf{U}$ The set of all motion **commands** or odometry (what the robot does)

2.1.3.2 3D Environment Representations

The map m generated by simultaneous localization and mapping (SLAM) must be structured efficiently to support unmanned aerial vehicle (UAV) decision-making tasks, including path planning, obstacle avoidance, and autonomous exploration. Common 3D environment representations include the following:

- **Point Clouds:** Point clouds are unstructured sets of 3D points $q_i \in \mathbb{R}^3$, typically generated by LiDAR sensors. While they offer high geometric accuracy, point clouds are memory-intensive and inefficient for spatial queries.
- **Occupancy Grids (Voxel Maps(11)):** Occupancy grids utilize uniform voxel grids to store binary or probabilistic occupancy states. Although these grids are straightforward to query, they do not scale efficiently in large, sparse environments because of their cubic memory complexity $O(n^3)$.
- **OctoMaps(12):** Multi-resolution 3D occupancy maps based on octree structures have been developed to address these limitations. These maps provide efficient $O(\log n)$ memory usage, explicitly represent free, occupied, and unknown space, and are widely used in UAV systems. Although they are less detailed than dense point clouds, octree-based maps achieve a practical balance between precision and computational efficiency.

2.2 Reinforcement Learning and Multi-Agent Decision-Making

2.2.1 Single-Agent Reinforcement Learning (RL)

Reinforcement Learning(13) is a branch of machine learning. In RL, an autonomous agent learns to make optimal decisions through repeated interaction with a dynamic environment. The goal is to select actions that maximize a long-term cumulative reward and improve performance over time.

Mathematical Framework: Markov Decision Process

A Reinforcement Learning problem is formally represented as a Markov Decision Process (MDP), defined by the tuple:

$$\mathcal{M} = (S, A, P, R, \gamma)$$

where:

- S is the set of possible environment states.
- A is the set of actions available to the agent.
- $P(s' | s, a)$ is the transition probability describing the environment's dynamics.
- $R(s, a) \in \mathbb{R}$ is the expected reward when taking action a in state s .
- $\gamma \in [0, 1]$ is the discount factor that weights future rewards.

At each timestep t , the agent observes a state $s_t \in S$, selects an action $a_t \in A$ according to its policy, receives a reward $r_{t+1} = R(s_t, a_t)$, and transitions to a new state $s_{t+1} \sim P(\cdot | s_t, a_t)$. (13)

Policy π

A *policy* defines the behavior of the agent. It is a mapping:

$$\pi : S \rightarrow \mathcal{P}(A),$$

where $\mathcal{P}(A)$ denotes probability distributions over the action set A . Thus, $\pi(a | s)$ represents the probability of taking action a in state s .

A deterministic policy is written as:

$$\pi(s) = a.$$

2.2 Reinforcement Learning and Multi-Agent Decision-Making

Reward and Return

The next component, the reward signal, quantifies the task objective. With this, the agent seeks to maximize its expected return, namely:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

Value Functions

Value functions are essential for evaluating the expected long-term utility of states and actions under a given policy.

State-value function:

$$V_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid s_t = s].$$

Action-value function:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid s_t = s, a_t = a].$$

Both value functions adhere to the Bellman equations, foundational to many RL algorithms. Reinforcement Learning algorithms are broadly organized based on what quantity they estimate, as follows:

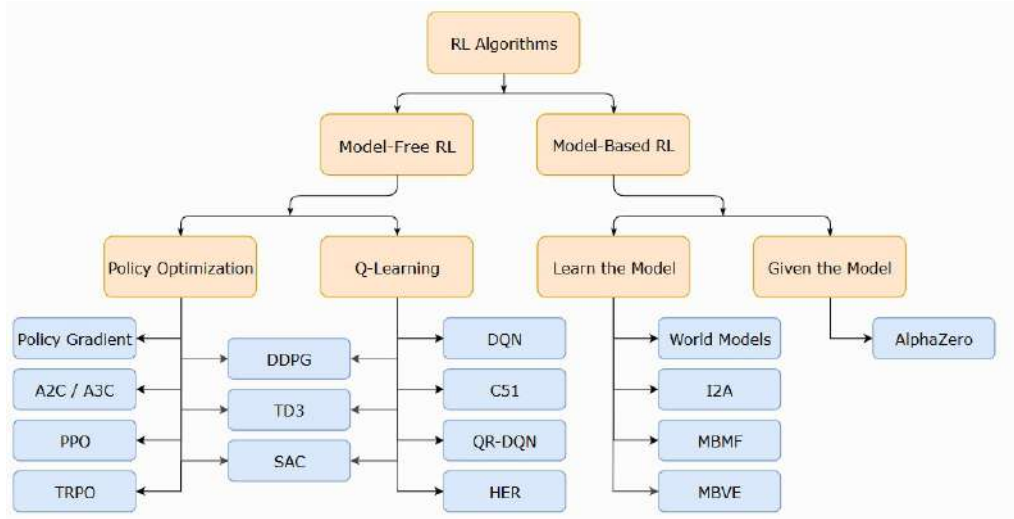


Figure 2.1: A non-exhaustive, but useful taxonomy of algorithms in modern RL. (Extracted from (1))

2.2 Reinforcement Learning and Multi-Agent Decision-Making

Beyond value-based algorithms, modern Reinforcement Learning methods often employ **policy gradients**, which directly optimize the parameters of a stochastic policy $\pi_\theta(a \mid s)$ by ascending the gradient of the expected cumulative reward.

Policy Gradient Formulation

The objective function in RL is to maximize the expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

According to the *Policy Gradient Theorem* (13), the gradient of this objective with respect to the policy parameters is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) Q_\pi(s_t, a_t)].$$

This provides the foundation for all **policy-based** algorithms, such as the classic **REINFORCE** method (14), which estimates the gradient via Monte Carlo sampling:

$$\nabla_\theta J(\theta) \approx \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t,$$

where G_t is the empirical return obtained after executing a trajectory.

Advantage Function and Variance Reduction

To reduce the variance of the gradient estimate, a baseline is introduced by subtracting the value of the current state $V_\pi(s)$ from the action-value estimate $Q_\pi(s, a)$. The resulting **advantage function** is defined as:

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s).$$

The corresponding gradient then becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) A_\pi(s_t, a_t)].$$

This modification forms the basis of all **Actor–Critic** algorithms, where:

- The **Actor** represents the policy π_θ , responsible for selecting actions.
- The **Critic** estimates either $V_\pi(s)$ or $Q_\pi(s, a)$ to compute the advantage signal guiding the policy update.

2.2 Reinforcement Learning and Multi-Agent Decision-Making

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

$$\phi \leftarrow \phi + \alpha_v \delta_t \nabla_\phi V_\phi(s_t),$$

$$\theta \leftarrow \theta + \alpha_\pi \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t,$$

where ϕ are critic parameters, θ the actor parameters, and α_v, α_π the learning rates.

To stabilize training, policy gradient methods often constrain the size of parameter updates.

Trust Region Policy Optimization (TRPO) (15) enforces a Kullback–Leibler (KL) divergence constraint between the old and new policy:

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} A_t \right] \quad \text{s.t.} \quad \mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)] \leq \delta.$$

Proximal Policy Optimization (PPO) (16) simplifies TRPO by introducing a *clipped surrogate objective*:

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right],$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$. This formulation prevents excessively large updates and has become fundamental for stable actor–critic training, motivating the development of several multi-agent extensions.

2.2.2 Multi-Agent Reinforcement Learning (MARL)

Multi-Agent Reinforcement Learning(2) extends classical reinforcement learning to scenarios in which multiple autonomous agents interact concurrently within a shared environment. While all agents occupy the same environment and evolve within a single global state, each agent maintains an individual policy and typically receives a distinct reward signal.

Core Challenges in MARL

The involvement of multiple adaptive agents introduces several fundamental challenges:

- **Non-stationarity:** As all agents update their behavior concurrently, the environment perceived by each agent changes over time. This dynamic violates the stationarity assumptions underlying most single-agent reinforcement learning methods, thereby complicating value estimation and convergence.

2.2 Reinforcement Learning and Multi-Agent Decision-Making

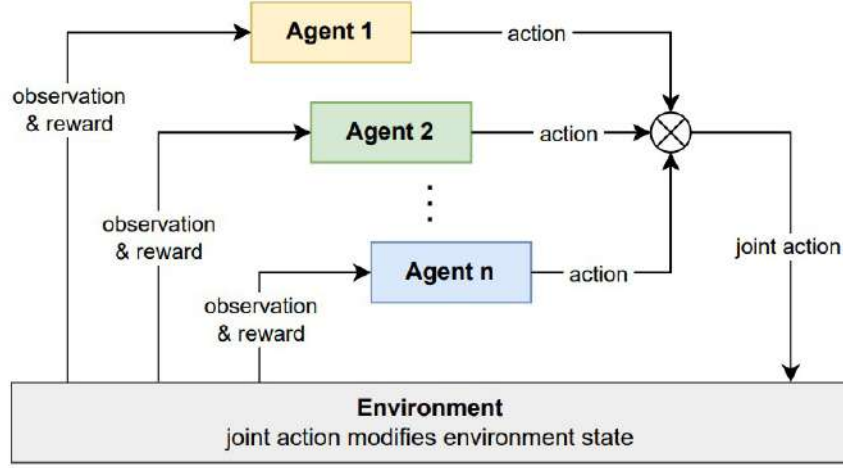


Figure 2.2: Schematic of multi-agent reinforcement learning. . (Extracted from (2))

- **Multi-agent credit assignment:** Outcomes frequently depend on the collective actions of all agents, making it difficult to determine the specific contribution of each agent to overall success or failure. This ambiguity complicates the learning process and slows policy improvement, particularly in cooperative settings.
- **Scalability and dimensionality:** The space of joint actions and interactions increases rapidly with the number of agents. This higher dimensionality complicates learning, coordination, and exploration, and may also destabilize training.
- **Communication:** In many scenarios, effective coordination requires information sharing among agents. Determining when, what, and how to communicate efficiently presents an additional challenge, particularly under bandwidth constraints or when only partial information is available.
- **Cooperative, adversarial, and mixed-motive interactions:** Agents may share goals, compete, or operate with partially aligned incentives. These different interaction regimes modify the strategic landscape, potentially leading to multiple equilibria, unstable behaviors, or convergence to suboptimal coordination patterns.

Algorithmic Paradigms

MARL algorithms are typically categorized into three broad groups, based on the primary learning objective:

2.2 Reinforcement Learning and Multi-Agent Decision-Making

- **Value-based methods:** These methods focus on estimating value functions for state-action pairs and improving behavior through value maximization. They aim to learn either joint or individual action-value estimates.
- **Policy-based methods:** These methods optimize agent strategies by making incremental adjustments based on performance, without explicitly calculating the value of each action.
- **Actor-critic methods:** These methods integrate value estimation (critic) with policy optimization (actor), leveraging both value-based and policy-based learning approaches.

The following figure or table provides a concise summary of these algorithmic categories.

2.2.3 Formal Models for Cooperative MARL

Markov Decision Processes (MDPs)

MDP is the classical model for single-agent reinforcement learning under full observability. The agent receives the full environment state s_t at each timestep and selects an action solely based on the current state, exploiting the Markov property. MDPs are adequate for standard RL but insufficient for multi-agent scenarios, where several agents interact concurrently.

Partial Observability

In many RL applications, the agent does not observe the full environment state but only receives a partial, noisy observation. This leads to the *Partially Observable Markov Decision Process* (POMDP), defined by:

$$\mathcal{P} = (S, A, O, P, Z, R, \gamma),$$

where O is the observation space and $Z(o \mid s, a)$ is the observation function. A POMDP is therefore the natural extension of an MDP when full observability is not available for a single agent.

Multi-Agent Partial Observability

When multiple agents interact under partial observability, the appropriate model is the *Partially Observable Stochastic Game* (POSG). The POSG generalizes both POMDPs

2.2 Reinforcement Learning and Multi-Agent Decision-Making

and stochastic games. Each agent i receives an individual observation, maintains its own policy, and attempts to maximize its own return. The POSG is the most general formal model used in MARL.

The progression of formal models can therefore be summarized as:

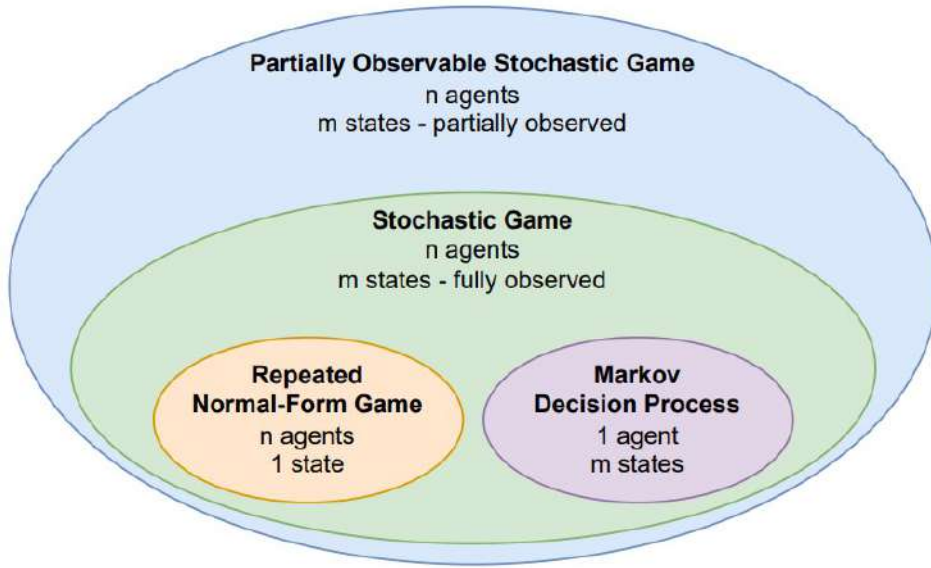


Figure 2.3: Hierarchy of game models . (Extracted from (2))

Decentralized POMDPs (Dec-POMDPs)

A Decentralized Partially Observable Markov Decision Process is a cooperative special case of the POSG in which all agents share a common reward function and must act in a decentralized manner(2).The Dec-POMDP is defined by the tuple:

$$\mathcal{D} = (I, S, \{A_i\}_{i \in I}, \{O_i\}_{i \in I}, P, Z, R, \gamma),$$

where:

- $I = \{1, \dots, N\}$ is the set of agents,
- S is the global state space,
- A_i and O_i are the action and observation spaces of agent i ,
- $P(s' \mid s, a_1, \dots, a_N)$ is the transition function,
- $Z(o_1, \dots, o_N \mid s', a_1, \dots, a_N)$ is the joint observation function,

2.2 Reinforcement Learning and Multi-Agent Decision-Making

- $R(s, a_1, \dots, a_N)$ is the common reward function,
- γ is the discount factor.

Each agent i receives a local observation o_t^i and must select its action a_t^i using only its own action–observation history:

$$h_t^i = (o_0^i, a_0^i, \dots, o_t^i).$$

Constrained Decentralized POMDPs

A Constrained Decentralized Partially Observable Markov Decision Process (C-Dec-POMDP) generalizes the standard Dec-POMDP framework by incorporating explicit constraints.

(17)

$$D_c = \left(I, S, \{A_i\}_{i \in I}, \{O_i\}_{i \in I}, P, Z, R, \gamma, \{C_k\}_{k=1}^K, \{d_k\}_{k=1}^K \right)$$

- I : finite set of agents.
- S : global state space of the environment.
- A_i : action space of agent i .
- O_i : observation space of agent i .
- $P(s' \mid s, \mathbf{a})$: transition function defining the probability of moving to state s' from state s under the joint action $\mathbf{a} = (a_1, \dots, a_N)$.
- $Z(\mathbf{o} \mid s', \mathbf{a})$: joint observation function giving the probability of agents receiving joint observation $\mathbf{o} = (o_1, \dots, o_N)$ after the transition.
- $R(s, \mathbf{a})$: shared reward function for cooperative agents.
- γ : discount factor, $\gamma \in [0, 1)$.
- $C_k(s, \mathbf{a})$: k -th cost function representing a constraint (e.g., resource usage, safety, communication).
- d_k : admissible upper bound for the cumulative discounted cost C_k .

The primary objective is to maximize the expected return subject to the satisfaction of all imposed constraints:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \mathbf{a}_t) \right] \quad \text{s.t.} \quad \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t C_k(s_t, \mathbf{a}_t) \right] \leq d_k, \quad \forall k.$$

3

Literature review

This chapter reviews prior work relevant to designing a fully autonomous and resilient multi-UAV system for 3D coverage in realistic urban environments under strict operational constraints such as energy, safety, no-fly zones, communication.

3.1 Classical Multi-UAV Coverage and Exploration

Coverage Path Planning (CPP) aims to determine a trajectory that visits all regions of a workspace while minimizing costs such as distance, time, or energy. In multi-agent contexts, Multi-Agent CPP extends this objective to a set of trajectories whose union ensures complete coverage. Given the NP-hardness of this problem (18), researchers frequently employ approximate and heuristic strategies. Traditional coverage planning for unmanned aerial vehicles (UAVs) employs deterministic decomposition and graph-based approaches. Grid, boustrophedon, and cell-based techniques guarantee complete area coverage in environments with known parameters. Recent research has modeled MCP as a Multiple-Set Traveling Salesman Problem (mTSP) with a minimum-energy objective and explicit energy consumption constraints, requiring each UAV to complete its designated region and return to base within its endurance limit(19).

In 3D urban spaces, voxel and octree representations are used by planners to account for altitude and occlusion, supporting volumetric coverage around complex structures.(11) Despite these advancements, such optimization-based approaches are typically offline and non-adaptive.

In addition to deterministic formulations, bio-inspired and heuristic optimisation algorithms, including Ant Colony Optimisation (ACO), Particle Swarm Optimisation (PSO), and Genetic Algorithms (GA), have been applied to enhance multi-UAV coordination and

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

coverage performance (20, 21). While these methods reduce overlap and improve path allocation, they rely on manually defined objective functions and static heuristics. As a result, they optimise only predefined criteria and do not enable the acquisition of adaptive strategies. This limitation has prompted the development of data-driven approaches in which UAVs learn cooperative behaviours through experience. Consequently, Multi-Agent Reinforcement Learning has emerged as a framework for autonomous coordination in uncertain environments.

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

To overcome these limitations, recent research has adopted MARL, enabling coordination to emerge organically rather than being predefined. In MARL, each UAV is treated as a learning agent that repeatedly interacts with its environment, receives feedback via a reward signal, and gradually adapts its policy. A well-known illustration is OpenAI’s “Hide and Seek” experiment (22), in which agents learned tool use, coordinated blocking, and sophisticated hiding strategies without these behaviours being explicitly programmed; they emerged purely from the interaction dynamics and the reward structure.

In cooperative multi-UAV settings, MARL algorithms differ mainly in the information they are allowed to use during *training* and during *execution*.

Centralized Training with Decentralized Execution (CTDE) has become the prevailing approach in recent deep Multi-Agent Reinforcement Learning research involving UAV swarms. This paradigm enables the use of comprehensive global information during training, such as 3D maps, joint UAV states, or communication graph structures. Importantly, CTDE ensures that the resulting policies can be executed onboard each UAV using only local sensing and limited communication. Formally, the cooperative multi-UAV control problem is modeled as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), a framework in which multiple agents operate without complete knowledge of the global state. (2)

$$\langle N, S, \{A_i\}, P, R, \{O_i\}, O, \gamma \rangle,$$

where N denotes the number of agents, S the global state space, A_i and O_i the action and observation spaces for agent i , P the transition model, $R(s, a)$ the joint reward function,

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

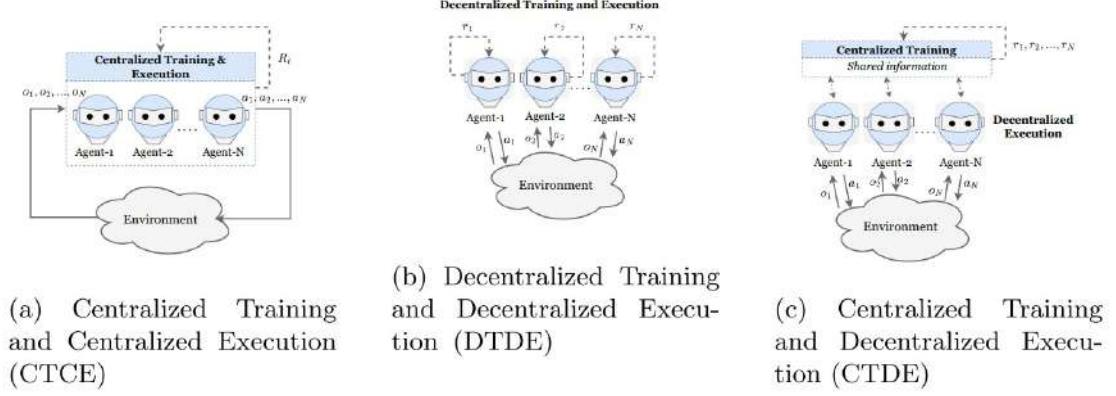


Figure 3.1: Comparison of Training and Execution Paradigms in MARL: a) CTCE is a fully centralized framework which combines all observations and actions of the agents into a joint observation-action space. b) DTDE is a term for fully decentralized setting that treats all agents independently with their own observation, action, and reward (23). c) CTDE framework has training in a centralized manner with information from other agent, then deploy the trained policy to each agent independently.(extracted from (3))

O the observation function, and γ the discount factor. The objective is to identify a set of decentralized policies

$$\pi = \{\pi_i(a_i | o_i)\}$$

that maximises the expected joint return:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

Within the CTDE paradigm, most deep MARL algorithms for cooperative control are categorized into many families:

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

Criterion	Value-Based MARL	Policy-Based MARL	Federated MARL
Core Idea	Learns joint or factorized value functions (e.g., Q-values)	Learns decentralized or centralized policies directly	Distributes learning across agents while preserving data locality
Representative Algorithms	QMIX, VDN, QTRAN, WQMIX	MAPPO, MADDPG, TRPO, HAPPO	FedRL, FedMARL, FedAvg+MARL
Action Space	Discrete	Continuous and discrete	Both (depends on local model)
Performance	High in structured/cooperative tasks	High in continuous, dynamic tasks	Moderate to high; depends on aggregation frequency and bandwidth
Environment	Best in fully observable or low-noise discrete settings	Works well in dynamic, partially observable, or continuous environments	Well-suited for distributed, privacy-sensitive, and bandwidth-limited environments
Scalability	Moderate (limited by value function complexity)	High (decentralized execution is effective)	High (supports large-scale distributed systems)
Communication Overhead	Low to moderate	Moderate (depends on policy sharing)	Low (only model parameters shared)
Training Stability	Sensitive to agent number and observability	More stable with proper exploration (e.g., GAE)	Depends on sync method and local training accuracy
Convergence Speed	Fast in simple settings	Moderate; requires more samples but is stable	Slower due to delayed aggregation and heterogeneity
Adaptability to Dynamic Environments	Low to moderate	High	High (supports model personalization)
Privacy Preservation	Not supported inherently	Not supported inherently	Strong (data remains local)
Suitability for UAV Applications	Suitable for cooperative discrete tasks (e.g., coverage)	Ideal for continuous control (e.g., formation, tracking)	Ideal for distributed UAV systems with privacy and bandwidth constraints

Figure 3.2: Expanded comparison of MARL learning techniques (Extracted from (3))

- **Centralised Critic Methods**

Policy-based CTDE methods train a decentralized policy (actor) for each agent, utilizing a centralized critic with access to global information, such as joint states or

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

joint observations, to stabilize training. This architecture is particularly well-suited for continuous action spaces, which frequently arise in UAV trajectory control.

MADDPG (Multi-Agent DDPG) (24) MADDPG adapts Deep Deterministic Policy Gradient (DDPG) for multi-agent environments. Each agent i possesses its own actor $Actor_i$ and a centralized critic $Q_i(s, a_1, \dots, a_N)$. The critic is trained using the actions of all agents.

MAPPO (Multi-Agent PPO) (25) A widely used baseline adapts Proximal Policy Optimization (PPO) by incorporating a centralized critic $V(s)$ to estimate the global state value, while actors are trained using local observations.

This approach explicitly addresses the credit assignment problem by employing a centralized critic $Q(s, a)$. For each agent i , it computes a counterfactual advantage function as follows:

$$A_i(s, a) = Q(s, a) - \sum_{a'_i} \pi_i(a'_i | o_i) Q(s, (a'_i, a_{-i})),$$

where a_{-i} represents the joint action of all agents except agent i . This formulation enables the policy gradient to isolate the individual contribution of each agent.

- **Value Decomposition Methods**

Value-based CTDE methods are typically applied in discrete action spaces. These methods aim to learn the global action-value function $Q_{exttot}(s, a)$ by decomposing it into per-agent utility functions $Q_i(o_i, a_i)$. Such decomposition is essential for decentralized execution, as it enables each agent to select actions greedily based on its local Q_i .

The primary challenge is to ensure that local greedy actions align with the global optimum, a property referred to as Individual-Global-Max (IGM).

VDN (10) (Value Decomposition Networks) The most straightforward approach assumes that the global Q-value is the sum of individual utility functions:

$$Q_{tot}(s, a) = \sum_{i=1}^N Q_i(o_i, a_i)$$

Although this additive decomposition is restrictive, it guarantees satisfaction of the IGM condition.

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

QMIX(2) A more advanced method generalizes VDN by employing a nonlinear mixing network that integrates per-agent utilities Q_i with the global state s :

$$Q_{\text{tot}}(s, a) = f_{\text{mix}}(Q_1(o_1, a_1), \dots, Q_N(o_N, a_N), s),$$

$$\text{subject to } \frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0, \quad \forall i$$

This constraint ensures that a local optimum for each Q_i corresponds to a local optimum for Q_{exttot} . As a result, decentralized execution is enabled, and a broader class of joint value functions can be represented compared to VDN.

Although these algorithms are essential for cooperative control, their implementation in real-world UAV swarms involves significant challenges. Major concerns include the design of effective reward signals, enforcement of operational constraints, and maintenance of communication within bandwidth and range limitations.

Communication in MARL

To address communication challenges, which are critical for decentralized execution, many contemporary architectures incorporate Graph Neural Networks. Local observations o_i are frequently inadequate for complex coordination tasks, such as resolving task conflicts to prevent redundancy. GNN-based methods represent the swarm as a dynamic graph $\mathcal{G}$, enabling agents to exchange and aggregate messages, such as hidden states, with neighboring agents. Architectures including CommNet (26), MAGNNET (27), and G2ANet (28) employ attention mechanisms to determine which information should be shared and with which agents, thereby optimizing bandwidth usage and enhancing coordinated decision-making.

Collaborative SLAM and Distributed Perception in UAV Swarms

Collaborative Simultaneous Localisation and Mapping (C-SLAM) provides the perceptual foundation for multi-UAV autonomy by enabling each agent to maintain a consistent and continuously updated environmental model. In this paradigm, UAV poses and observations are represented as nodes and edges in a factor graph, which is optimised to jointly estimate both the global map and the trajectories of all participating agents (29). Each UAV performs local SLAM and exchanges compact information, such as keyframes or candidate loop closures, with neighbouring agents to minimise global reprojection or scan-matching errors while operating under limited communication and computational resources.

3.2 Multi-Agent Reinforcement Learning for UAV Control and Communication

The distributed SLAM problem can be formulated as the following factor-graph optimisation:

$$x^* = \arg \min_x \sum_{(i,j) \in \mathcal{E}} \|z_{ij} - h_{ij}(x_i, x_j)\|_{\Sigma_{ij}^{-1}}^2, \quad (3.1)$$

In this formulation, x_i and x_j denote UAV poses, z_{ij} represents their relative measurements, and $\mathcal{E}$ comprises the set of intra-robot and inter-robot constraints. Maintaining estimation consistency and scalability as $\mathcal{E}$ grows during exploration remains a significant challenge.

Early decentralised systems, such as *Swarm-SLAM*, employ a sparse peer-to-peer architecture that integrates multiple sensors (LiDAR, stereo, RGB-D, IMU) and selectively exchanges only the most informative loop closures. The loop-closure selection mechanism aims to maximise the *algebraic connectivity* of the pose graph, defined as the second smallest eigenvalue $\lambda_2(L)$ of its Laplacian matrix L , which quantifies network robustness.

The optimisation objective is given by:

$$\max_{\mathcal{E}'} \lambda_2(L(\mathcal{E}')) \quad (3.2)$$

subject to a communication budget constraint

$$|\mathcal{E}'| \leq B, \quad (3.3)$$

Here, $\mathcal{E}'$ denotes the selected subset of loop closures and B specifies the maximum allowed bandwidth. By constraining message exchange, Swarm-SLAM preserves global map consistency while reducing redundant communication. Robust solvers, such as Graduated Non-Convexity (GNC), further ensure reliable convergence in the presence of noise and outliers.

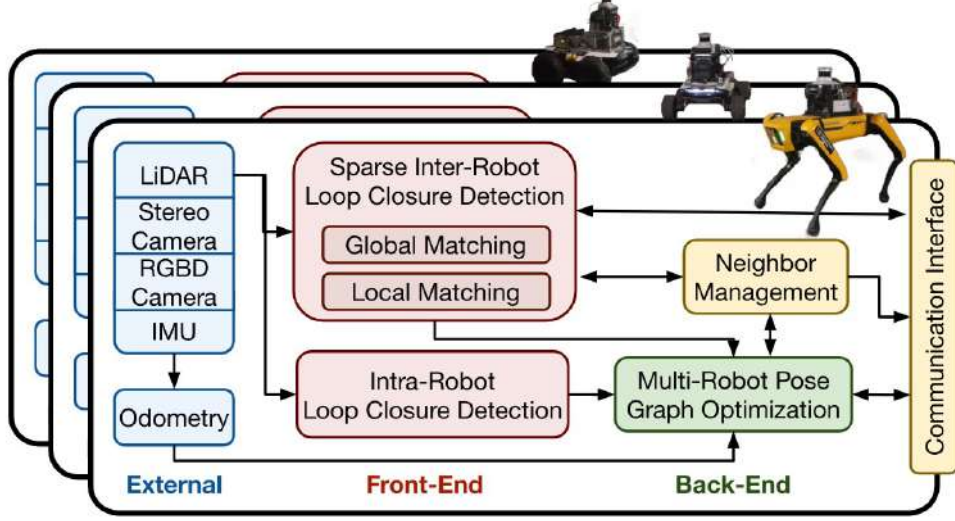


Figure 3.3: General architecture of Swarm-SLAM showing local pose-graph estimation, selective loop-closure exchange, and distributed map optimisation. (Extracted from (4))

In summary, the principles of sparse communication, spectral graph optimisation, and robust distributed solvers form the core perception layer that enables the integration of multi-agent reinforcement learning (MARL) for coordinated, decentralised decision-making in UAV swarms.

3.3 Reward Design, Constraints, and Safe Reinforcement Learning

In multi-UAV three-dimensional missions, the reward function is generally constructed as a weighted sum of multiple objectives. (2)

Limitations and Potential-Based Shaping. This manually tuned weighted-sum approach presents two significant limitations. First, it is brittle, as the relative tuning of weights such as w_{cov} versus w_{coll} , is difficult and highly sensitive: a small collision penalty may be ignored, whereas a large one may lead to overly cautious agents. Second, the reward signal is often sparse; an agent may receive $R_{\text{cov}} = 0$ for many timesteps while navigating toward unexplored areas, reducing learning efficiency.

To mitigate reward sparsity, Potential-Based Reward Shaping (PBRS) (30) offers a formal mechanism to introduce a dense intermediate reward F without altering the optimal

3.3 Reward Design, Constraints, and Safe Reinforcement Learning

policy. The shaped reward R' is defined as:

$$R'(s, a, s') = R(s, a, s') + \gamma\Phi(s') - \Phi(s), \quad (3.4)$$

where $\Phi(s)$ is a potential function over states.

Safe MARL and Constrained MDPs

The most critical limitation of reward-based approaches is safety. Encoding safety as a large negative reward (via $w_{\text{coll}}C_i^t$) is fundamentally insufficient for real-world systems. Standard RL algorithms rely on trial-and-error and must experience failures to learn to avoid them; however, even a single collision is unacceptable in physical UAV systems.

This limitation motivates a transition from standard Decentralized Partially Observable Markov Decision Processes (Dec-POMDPs) to Constrained Multi-Agent Reinforcement Learning. A CMDP extends the MDP framework with M cost functions $C_k(s, a)$ and associated budget thresholds d_k (31).

The objective is no longer simply to maximize return, but to maximize it *subject to* safety constraints. The goal is to find a policy π that:

$$\max_{\pi} J_R(\pi) = \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right], \quad (3.5)$$

$$\text{subject to} \quad J_{C_k}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t C_k(s_t, a_t) \right] \leq d_k, \quad \forall k \in \{1, \dots, M\}. \quad (3.6)$$

In a UAV context, a cost C_k may represent undesirable events: for instance $C_{\text{coll}}(s, a) = 1$ if action a results in a collision or distance violation, and 0 otherwise, with a strict budget $d_{\text{coll}} = 0$. Similarly, $C_{\text{energy}}(s, a)$ may denote the energy consumed, with budget d_{energy} tied to battery capacity.

Lagrangian Methods and Safe Algorithms. The most common approach to solving CMDPs is Lagrangian relaxation, which converts the constrained problem into an unconstrained dual problem by introducing Lagrange multipliers $\lambda = \{\lambda_k \geq 0\}_{k=1}^M$. The resulting Lagrangian objective is:

$$\mathcal{L}(\pi, \lambda) = J_R(\pi) - \sum_{k=1}^M \lambda_k (J_{C_k}(\pi) - d_k). \quad (3.7)$$

Learning proceeds as a min-max game: the policy π maximizes $\mathcal{L}$, while the multipliers λ are updated via gradient ascent:

$$\lambda_k^{t+1} = \max\left(0, \lambda_k^t + \eta (J_{C_k}(\pi_t) - d_k)\right). \quad (3.8)$$

3.3 Reward Design, Constraints, and Safe Reinforcement Learning

This principle underpins many Safe MARL algorithms. For example, MAPPO-Lagrangian and Safe-MAPPO (32) incorporate Lagrangian relaxation into the centralized training with decentralized execution (CTDE) framework: decentralized actors π_i optimize $\mathcal{L}$, while a centralized critic estimates both the expected return V_R and expected costs V_{C_k} for low-variance updates. MAPCO further extends this framework via a cost-constrained advantage function to improve stability (33).

Pointwise Constraints and Stability in Multi-Agent Reinforcement Learning with Control Barrier Functions

In contrast to Lagrangian methods, which address safety as an aggregated constraint over an entire episode, the use of Control Barrier Functions (CBF) and Control Lyapunov Functions (CLF) seeks to ensure safety and stability deterministically at each time step (34, 35). This approach reformulates the safe control problem as a constrained optimization program that is solved in real time. To maintain the forward invariance of a safe set defined by $h(s) \geq 0$, the controller must satisfy the condition $L_f h(s) + L_g h(s)a + \alpha(h(s)) \geq 0$ at every time step. This constraint ensures that the derivative of the barrier function does not decrease too rapidly toward unsafe regions (35). Concurrently, stability and goal progression are governed by a Lyapunov function $V(s)$, with the CLF constraint enforcing an exponential decay of the system’s energy (35).

Recent research integrates these control-theoretic guarantees with Multi-Agent Reinforcement Learning (MARL) to overcome the limitations of exclusively model-based or learning-based methods. Within the HMARL-CBF framework, the reinforcement learning agent does not directly learn the action. Instead, it learns the parameters ϕ of the optimization problem, such as the coefficients of the class $\mathcal{K}$ function or the geometry of safety zones, using partial observations (35). Alternatively, the Layered Safe MARL architecture utilizes reinforcement learning for strategic planning to avoid situations where constraints from multiple agents become unsolvable simultaneously (34). In this layered architecture, the CBF or Control Barrier Value Function (CBVF) filter acts as a tactical safety layer to prevent imminent collisions, enabling the reinforcement learning policy to acquire efficient behaviors without excessive conservatism (34).

3.4 A Review of Simulation Platforms for Reinforcement Learning in UAV Applications

3.4 A Review of Simulation Platforms for Reinforcement Learning in UAV Applications

Simulators are essential for training multi-agent reinforcement learning (MARL) algorithms in unmanned aerial vehicle coverage tasks, as they address challenges related to safety, cost, and low sample efficiency in real-world experimentation. The selection of a simulation platform involves balancing training speed with Sim-to-Real fidelity. GPU-accelerated environments, such as Isaac Gym, facilitate rapid policy exploration through large-scale parallelization of agents(5). As deployment nears, realistic validation becomes increasingly important, necessitating high-fidelity simulation of sensors such as LiDAR and RGB cameras, as well as accurate noise models, environmental interactions, and communication constraints. AirSim offers photorealistic rendering and advanced sensing capabilities, while Gazebo/ROS is widely regarded as the standard for robotics integration and system-level testing. Although MATLAB/Simulink remains prevalent in aerospace research for high-fidelity control modeling, open-source platforms like Gazebo/ROS and AirSim are generally favored in MARL research due to their flexibility, extensibility, and capacity to represent complex three-dimensional interactive environments.

COMPARISON OF FEATURES FOR WIDELY USED UAV SIMULATORS: INCLUDED (✓), PARTIALLY INCLUDED (*), AND NOT INCLUDED (✗)

Simulator	Physics Engine	Rendering	OS			Interfaces	(S/H)ITL	License	Open-Source	Active	Ref.
			L	W	M						
Gazebo Classic	ODE, Bullet, DART, Simbody	OGRE	✓	*	✓	ROS 1/2, C++, RL	PX4, ArduPilot, CF	Apache-2.0	✓	✓	[20]
Gazebo	Bullet, DART, TPE	OGRE	✓	*	✓	ROS 1/2, C++, Python, RL	PX4, ArduPilot, CF	Apache-2.0	✓	✓	[25]
Isaac (Pegasus, Aerial Gym)	NVIDIA PhysX, Flex	Vulkan	✓	✗	✗	ROS 1/2, Python, RL	Pegasus: PX4	Proprietary (BSD 3)	✗ (✓, ✓)	✓	[28]–[31]
Webots	ODE	OpenGL	✓	✓	✓	ROS 1/2, C/C++, Python, MATLAB, Java	ArduPilot, CF	Apache-2.0	✓	✓	[32]
CoppeliaSim	Bullet, ODE, Vortex, Newton, MuJoCo	OpenGL	✓	✓	✓	ROS 1/2, C/C++, Python, MATLAB, Java, Lua, Octave	—	GNU GPL, Commercial	*	✓	[35]
AirSim	NVIDIA PhysX	Unreal, Unity	✓	✓	✓	ROS 1, C++, Python, C#, Java, RL	PX4, ArduPilot	MIT	✓	✗	[38]
Flightmare	Ad hoc, Gazebo Classic	Unity	✓	✗	✗	ROS 1, C++, RL	—	MIT	✓	✗	[40]
FlightGoggles	Ad hoc	Unity	✓	*	✗	ROS 1, C++	Motion Capture	MIT	✓	✗	[42]
gym-pybullet-drones	PyBullet	OpenGL	✓	*	✓	Python, RL	Betaflight, CF	MIT	✓	✓	[50]
RotorTM	Ad hoc	OpenGL	✓	✗	✗	ROS 1, Python, MATLAB	—	GNU GPL	✓	✓	[54]
MATLAB UAV Toolbox	MATLAB	Unreal	✓	✓	✓	ROS 2, MATLAB	PX4	Proprietary, Commercial	✗	✓	[55]

Figure 3.4: Comparative review of widely used UAV simulators.(extracted from (5))

3.4 A Review of Simulation Platforms for Reinforcement Learning in UAV Applications

Figure 3.4 summarizes the primary characteristics of the simulators most frequently cited in the literature.

Research gap

Despite notable advancements, existing methods typically lack a fully decentralized architecture. These methods do not integrate distributed perception, SLAM-based map fusion, and local communication via dynamic graph networks. Additionally, existing approaches do not comprehensively address persistent and resilient exploration under energy, communication, and regulatory constraints in complex, partially observable three-dimensional urban environments. The present study introduces a unified multi-agent reinforcement learning (MARL) framework. This framework is designed to address these challenges with enhanced autonomy, scalability, and robustness.

4

Proposed Methodology and System Architecture

This work builds on safe MARL approaches such as MAPPO-Lagrangian (32) and MAPCO (33), and communication-based coordination models like MAGNNET (27). Unlike prior methods that treat safety, perception, and coordination separately, we combine decentralized SLAM (29) with control-theoretic safety layers (34, 35). Hard constraints enforce collision avoidance, geofencing, and energy limits. The framework is validated in Gazebo for robust multi-UAV exploration in GPS-denied environment.

4.1 Proposed Approach

This study investigates the deployment of a UAV swarm for autonomous mapping and exploration of large-scale urban environments. The proposed approach is persistent, constraint-aware, and decentralized. Each UAV is equipped with onboard LiDAR, an inertial measurement unit (IMU), and limited-range communication modules. The UAVs are required to avoid restricted airspaces, adhere to energy budgets, and operate under communication and GPS limitations.

The primary features of the mission are as follows:

- Real-time decentralized mapping using local SLAM,
- Peer-to-peer data exchange based on proximity,
- Persistent operation through autonomous recharging,
- Robustness to GPS-denied regions and bandwidth limitations.

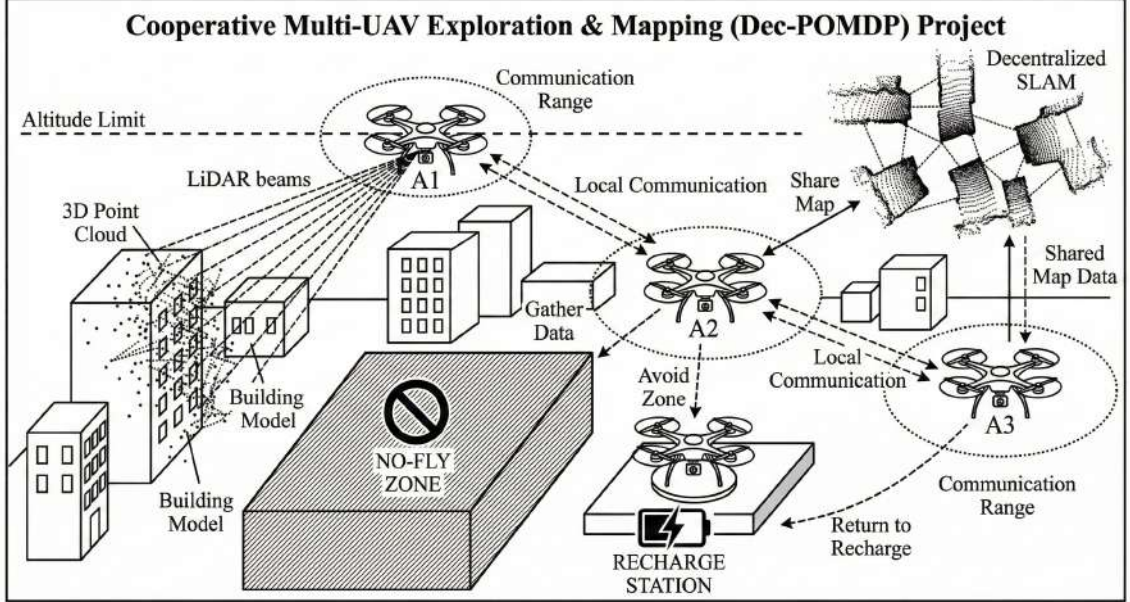


Figure 4.1: Proposed decentralized multi-UAV exploration and mapping architecture.

4.2 Formal Model and Constrained Problem Formulation

The UAV swarm operates under a strict set of operational constraints, including limited battery capacity, bounded onboard storage, altitude restrictions, collision-avoidance requirements, short-range communication, GPS-denied conditions, and legally enforced no-fly zones.

To formalize this problem, we adopt a Decentralized Partially Observable Markov Decision Process (**Dec-POMDP**), defined as:

$$D = (I, S, A_i, O_i, P, Z, R, \gamma) \quad (4.1)$$

The elements of this tuple are:

- I : Set of agents (UAVs) with $N = |I|$,
- S : Global state space including UAV and environment states,
- A_i : Action space of agent i ,
- O_i : Observation space of agent i ,
- P : State transition function,

4.2 Formal Model and Constrained Problem Formulation

- Z : Observation probability function,
- R : Global reward function shared by the agents,
- γ : Discount factor.

Each UAV functions under partial observability, utilizing local sensor data and intermittent communication with proximate agents.

4.2.1 Operational Constraints

Energy limits and autonomous return. UAVs have limited battery capacity, and energy consumption increases with motion and maneuvering. This imposes a hard safety requirement:

$$Q_i(t) > Q_{\min} \Rightarrow \text{continue mission}, \quad Q_i(t) \leq Q_{\min} \Rightarrow \text{return to base.} \quad (4.2)$$

This constraint is essential for UAV survival and supports persistent, long-term operation.

Limited onboard storage. Each UAV stores a bounded amount of local SLAM data (keyframes and LiDAR scans). When memory reaches its limit, the UAV must stop exploration and return to upload data:

$$W_i(t) \geq W_{\max} \Rightarrow \text{offload map at base.} \quad (4.3)$$

This requirement ensures continuous and reliable 3D reconstruction.

Collision avoidance and no-fly zones. Urban airspaces contain obstacles, buildings, and legally restricted volumes. These constraints guarantee both operational safety and regulatory compliance:

$$p_i(t) \notin \mathcal{Z}, \quad \|p_i(t) - p_j(t)\| > d_{\text{safe}}, \quad \forall j \neq i. \quad (4.4)$$

The set $\mathcal{Z}$ corresponds to *forbidden voxels* explicitly encoded in the OctoMap, representing regulatory no-fly zones and unsafe regions that UAVs must avoid at all times. Violations incur strong penalties in the reward function.

Communication range. Communication is local and opportunistic. Two UAVs can exchange SLAM information only if their distance is within a limited range:

$$\|p_i(t) - p_j(t)\| \leq r_c. \quad (4.5)$$

This naturally induces a dynamic communication graph.

4.2 Formal Model and Constrained Problem Formulation

GPS-denied conditions. Urban canyons degrade GNSS signals, making global localization unreliable. UAVs therefore estimate their pose using onboard SLAM:

$$\hat{x}_i(t) = f_{\text{SLAM}}(\text{LiDAR}_i(t), \text{IMU}_i(t)). \quad (4.6)$$

This reinforces the need for decentralized mapping.

Altitude restrictions. Civil rules impose vertical limits:

$$h_{\min} \leq z_i(t) \leq h_{\max}. \quad (4.7)$$

Persistence and resilience. Each UAV independently maintains its own SLAM, mapping, and decision-making modules, ensuring that the system remains functional even if individual agents fail. The **explore-return-upload-resume** cycle supports uninterrupted and scalable system operation.

Collectively, these constraints justify the adoption of a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) formulation and guide the design of decentralized SLAM, communication-limited coordination, and constraint-aware MARL policies.

4.2.2 Constrained Optimization Objective

The goal is to learn decentralized policies:

$$\pi = \{\pi_i(a_i \mid o_i)\}_{i=1}^N, \quad (4.8)$$

that maximize the expected long-term reward:

$$\max_{\pi} J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(t) \right], \quad (4.9)$$

subject to:

$$\begin{aligned} p_i(t) &\notin \mathbb{Z}, \\ \|p_i(t) - p_j(t)\| &> d_{\text{safe}}, \\ h_{\min} &\leq z_i(t) \leq h_{\max}, \\ Q_i(t) &> 0, \\ W_i(t) &< W_{\max}. \end{aligned} \quad (4.10)$$

This defines the complete constrained Dec-POMDP optimization problem governing persistent, safe, decentralized multi-UAV exploration and mapping.

4.3 Learning Architecture

To enable persistent exploration in a constrained, continuous 3D environment, we propose a learning architecture based on the **centralized training paradigm with Decentralized Execution**.

4.3.1 Algorithmic Selection

We utilize **MAPPO** (Multi-Agent Proximal Policy Optimization) rather than MADDPG to ensure training stability in high-dimensional continuous spaces. Unlike deterministic policies that require artificial noise injection, MAPPO learns a stochastic Gaussian policy π_θ parameterized by a mean μ and a standard deviation σ . The action a_t is sampled using the reparameterization trick:

$$a_t = \tanh(\mu_\theta(o_t)) + \sigma_\theta(o_t) \cdot \xi, \quad \xi \sim \mathcal{N}(0, I) \quad (4.11)$$

The tanh activation constrains the mean to the feasible control interval $[-1, 1]$, while the learnable σ_θ enables smooth, self-annealing exploration essential for flight dynamics.

The execution flow of the algorithm is detailed in Algorithm 1. It explicitly highlights the CTDE mechanism: actors collect data in a decentralized manner using local observations, while the critic optimizes the policy using global information to stabilize the learning process against non-stationarity.

Algorithm 1 Multi-Agent PPO with Centralized Training (CTDE)

```

1: Initialize decentralized Actor parameters  $\theta_i$  for each agent  $i \in \{1..N\}$ 
2: Initialize Centralized Critic parameters  $\phi$ 
3: Initialize Replay Buffer  $\mathcal{B}$ 
4: for episode = 1 to  $M$  do
5:   Reset environment, observe initial global state  $s_0$  and observations  $\mathbf{o}_0$ 
6:   for step  $t = 0$  to  $T$  do
7:     for agent  $i = 1$  to  $N$  do
8:       Get local observation  $o_{i,t}$  via GNN embedding
9:       Sample action  $a_{i,t} \sim \pi_{\theta_i}(o_{i,t})$  ▷ Decentralized Execution
10:    end for
11:    Execute joint action  $\mathbf{a}_t$ 
12:    Observe rewards  $\mathbf{r}_t$  and next state  $s_{t+1}$ 
13:    Store  $(s_t, \mathbf{o}_t, \mathbf{a}_t, \mathbf{r}_t, s_{t+1})$  in  $\mathcal{B}$ 
14:  end for

15:  Centralized Training Phase
16:  Compute global value  $V_\phi(s_t)$ 
17:  Compute advantages  $\hat{A}_{i,t}$  using GAE( $\lambda$ )
18:  Compute returns  $\hat{R}_{i,t} = \hat{A}_{i,t} + V_\phi(s_t)$ 
19:  for epoch = 1 to  $K$  do
20:    Update Critic
21:    Minimize

$$\mathcal{L}(\phi) = \frac{1}{NT} \sum_{i,t} \left( V_\phi(s_t) - \hat{R}_{i,t} \right)^2$$


22:    Update Actors
23:    Compute ratio

$$r_t(\theta_i) = \frac{\pi_{\theta_i}(a_{i,t}|o_{i,t})}{\pi_{\theta_{old}}(a_{i,t}|o_{i,t})}$$


24:    Maximize PPO objective

$$\mathcal{L}(\theta_i) = \min \left( r_t \hat{A}_{i,t}, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right)$$


25:  end for
26:  Clear  $\mathcal{B}$ 
27: end for

```

4.3.2 Graph-Based Communication and Swarm Coordination

Standard Reinforcement Learning does not work in a decentralized UAV swarm because each agent sees the environment as changing all the time. The state transitions depend on the actions of other agents that aren't seen. We model the whole swarm interaction as a **single dynamic communication graph** $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$ to solve this without a central controller. In this structure, the vertices $\mathcal{V}$ represent the agents and the edges $\mathcal{E}_t$ exist between the agents separated by a distance less than the communication radius R_{comm} . This single graph serves as the global medium for **Message passing**, allowing local information to propagate across the fleet.

Crucially, **aggregating neighbor information** using a simple mean or max-pooling operation is insufficient for navigation tasks. To address this, we employ a **Graph Attention Network (GAT)**. This mechanism allows the agent to dynamically learn which **neighbors require attention** based on their relative state. The attention coefficient α_{ij} , representing the importance of message h_j from neighbor j to agent i , is computed with a softmax over the neighborhood $\mathcal{N}_i$:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}h_i \parallel \mathbf{W}h_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}h_i \parallel \mathbf{W}h_k]))} \quad (4.12)$$

where $\mathbf{W}$ is a learnable weight matrix transforming the input features, and $\parallel$ denotes concatenation. This ensures that $\sum_j \alpha_{ij} = 1$, making the aggregation invariant to the scale of input features. The final context embedding c_i is then computed as a weighted sum, filtering out noise while amplifying critical interactions:

$$c_i = \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W}h_j \right) \quad (4.13)$$

Integration with MAPPO Observation Space

This graph-based mechanism fundamentally alters the observation space used by the MAPPO algorithm. Instead of relying solely on raw sensor data o_{raw} , the policy input is the **augmented observation** $\tilde{o}_i$:

$$\tilde{o}_i = [o_{raw} \parallel c_i] \quad (4.14)$$

By conditioning the policy on this permutation-invariant context vector c_i , we transform the non-stationary multi-agent problem into a quasi-stationary problem from the perspective of the local agent. The MAPPO actor $\pi_\theta(a_i|\tilde{o}_i)$ thus receives a "context-aware" state

description, enabling it to anticipate neighbor movements without requiring explicit joint-action modeling. This architecture ensures **scalability**. The trained policy relies on local density rather than specific agent identities, allowing a model trained with 3 drones to be deployed zero-shot on a swarm of 10 or more agents.

4.3.3 Hybrid Constraint Management Architecture

To satisfy the operational requirements, we implement a **Hybrid Safety Architecture**. This system operates on two distinct timescales consisting of a probabilistic outer loop using Lagrangian Relaxation for cumulative constraints and a deterministic inner loop using Control Barrier Functions for hard safety.

4.3.3.1 Lagrangian Relaxation for Cumulative Constraints

Operational constraints such as battery life $Q_i(t)$ and storage capacity $W_i(t)$ are cumulative. We formulate these quantities as soft constraints within a Constrained Markov Decision Process. The limits defined in 4.2 and 4.3 are incorporated into the reward function by using learnable Lagrange multipliers. The augmented reward function is defined as:

$$R'(\pi) = R_{mission} - \lambda_{energy} \cdot \max(0, Q_{min} - Q_i(t)) - \lambda_{data} \cdot \max(0, W_i(t) - W_{max}) \quad (4.15)$$

The multipliers λ are updated using dual ascent at a slower timescale than the policy parameters θ :

$$\lambda_{k+1} = \max(0, \lambda_k + \alpha \cdot \text{ConstraintViolation}_k) \quad (4.16)$$

4.3.3.2 Control Barrier Functions for Hard Safety

Collision avoidance and altitude restrictions allow for zero violation tolerance. We employ a Control Barrier Function filter that projects the neural network action u_{RL} a safe set $\mathcal{C}$ defined by linear constraints.

Quadratic Program with Slack Variables

In dense urban environments, the feasible control space may become empty due to conflicting constraints such as being trapped between a No-Fly Zone and the ceiling. To prevent solver failure, we introduce a relaxation vector δ . The final control input u^* is obtained by solving the following robust Quadratic Program at each control cycle:

$$u^* = \underset{u, \delta}{\operatorname{argmin}} \left(\frac{1}{2} \|u - u_{RL}\|^2 + \sum_j P \cdot \delta_j^2 \right) \quad (4.17)$$

This optimization is subject to the relaxed forward invariance conditions for each barrier $h_j(x)$:

$$\nabla h_j(x)^T u \geq -\gamma h_j(x) - \delta_j \quad (4.18)$$

Here, u represents the control vector $[v_x, v_y, v_z, \omega]^T$ and P is a large penalty scalar ensuring that relaxation δ_j remains non-zero only when strictly necessary to avoid infeasibility.

4.3.3.3 Analytical Safety

We explicitly select the Analytical Approach over Learned Control Barrier Value Functions for two reasons. First, learned modules approximate the barrier function using neural networks, which can lead to errors in unknown states. Our analytical approach derives the barrier directly from the physics of the drone, providing a provable safety certificate. Second, by using a pre-computed analytical filter, the Reinforcement Learning agent does not need to explore unsafe regions, which accelerates the training process.

Barrier Implementations

We explicitly derive the linear constraints for the operational limits defined in Section 4.2.

Altitude Geofencing For the Minimum Altitude Constraint where $h_1(x) = z - h_{min} \geq 0$, the gradient corresponds to the unit vector along the Z-axis. The constraint strictly limits downward velocity near the floor:

$$\underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}}_{\nabla h_1(x)^T} \cdot u \geq -\gamma(z - h_{min}) - \delta_1 \quad (4.19)$$

For the Maximum Altitude Constraint where $h_2(x) = h_{max} - z \geq 0$, the gradient directs velocity downwards:

$$\underbrace{\begin{bmatrix} 0 & 0 & -1 & 0 \end{bmatrix}}_{\nabla h_2(x)^T} \cdot u \geq -\gamma(h_{max} - z) - \delta_2 \quad (4.20)$$

No-Fly Zone Avoidance Static restricted areas are modeled as cylindrical No-Fly Zones. For a zone centered at (c_x, c_y) with radius R , the barrier function is $h_{obs}(x) = (x - c_x)^2 + (y - c_y)^2 - R^2$. The safety filter enforces a constraint on the planar velocity components:

$$\underbrace{\begin{bmatrix} 2(x - c_x) & 2(y - c_y) & 0 & 0 \end{bmatrix}}_{\nabla h_{obs}(x)^T} \cdot u \geq -\gamma \left((x - c_x)^2 + (y - c_y)^2 - R^2 \right) - \delta_{obs} \quad (4.21)$$

This ensures the velocity vector always possesses a component pointing away from the forbidden zone boundary.

4.3.4 Reward Function Design and Multi-Agent Credit Assignment

The reward function balances mission objectives with physical safety using a hybrid structure of shared global rewards and individual local penalties. We explicitly utilize fixed weights instead of Potential-Based Reward Shaping (PBRs) because defining valid potential functions for 3D geometric mapping in non-linear urban canyons is mathematically complex and often leads to vanishing gradients. Following the benchmarks of (36), dense rewards with fixed scaling provide a more stable learning signal and establish a strict hierarchy where survival is prioritized over mapping ($w_{coll} \gg w_{exp}$).

Coordination relies on a **shared discovery reward** where each new voxel added to the OctoMap benefits the whole fleet, promoting natural task division. Collisions and boundary violations are treated as **individual penalties** applied only to the responsible drone, preventing the entire swarm from slowing down due to a single failure. The local reward $R_i(t)$ combines mission objectives, stability terms, and adaptive Lagrangian penalties λ_k .

$$R_i(t) = w_{exp} \Delta V_{global} + w_{surv} + w_{stab} + w_{time} - \sum_k \lambda_k C_k(i, t) \quad (4.22)$$

Table 4.1: Fixed Reward Weights for Swarm Convergence

Parameter	Value	Mission Function
w_{exp}	5.0	Shared reward per new voxel added to the map
w_{time}	-0.05	Step penalty to encourage time efficiency
w_{coll}	-500.0	Local penalty for physical drone collisions
w_{bound}	-1000.0	Local penalty for leaving authorized workspace
$w_{offload}$	200.0	Bonus for successful data delivery at base
w_{stab}	-0.5	Penalty for excessive roll or pitch magnitude
w_{surv}	0.1	Constant bonus for maintaining safe flight

Table 4.1 summarizes the hyperparameter values used to ensure stable convergence in complex environments.

This chapter has established a comprehensive framework for decentralized multi-UAV exploration by integrating constrained Dec-POMDP modeling with MAPPO and Graph Attention Networks. By combining Lagrangian relaxation for long-term mission persistence with Control Barrier Functions for instantaneous safety, the proposed architecture ensures collision-free navigation while respecting strict operational limits in complex environments.

5

Implementation and Experimental Setup

5.1 Simulation Environment

5.1.1 Physics Engine and Procedural World

The primary simulation backend is **Gazebo Harmonic**, chosen for its high-fidelity physics and native ROS 2 compatibility. The physics server runs headlessly to maximize resources for neural network training. To ensure generalization, the environment is generated procedurally using a fixed random seed, creating a 200×200 m urban canyon populated with randomized buildings and No-Fly Zone cylinders.

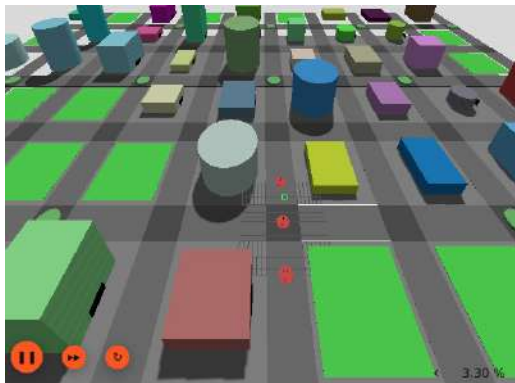


Figure 5.1: Simulation procedurally generated 3D urban canyon in Gazebo Harmonic

The simulation time is **step-locked** (50 ms ticks), guaranteeing absolute determinism and synchronization between the RL policy actions and physics updates.

5.1.2 UAV Swarm and Hardware Modeling

The swarm consists of three **X500-class quadcopters**, initialized at 60% of their battery capacity to force early decision-making regarding energy. Each drone is equipped with a specific payload of sensors:

- **3D LiDAR:** Forward obstacle detection and spatial mapping.
- **Downward LiDAR:** GPU ray-cast scan for ground coverage tracking.
- **IMU & Odometry:** High-frequency kinematic state estimation.

Beyond static sensing, the simulation incorporates strict operational dynamics governed by resource management constraints.

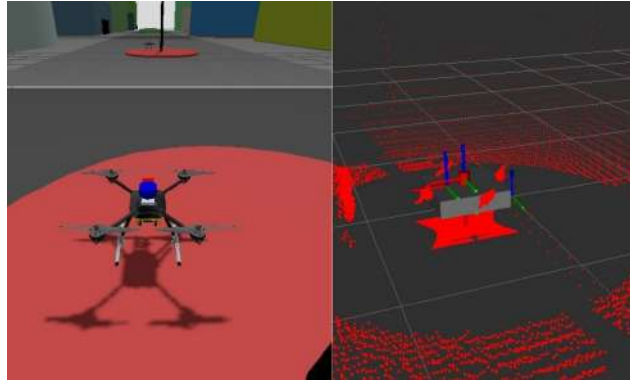


Figure 5.2: Simulated X-frame multi-rotor UAV with onboard sensors.

Each UAV is assigned to a dedicated ground station. During exploration, the onboard battery gradually depletes while the local storage progressively fills with mapped voxel data. To ensure sustained operation, the UAV must autonomously return to its designated station whenever resource levels reach critical thresholds. Upon successful docking, the battery is fully recharged, the onboard storage is cleared, and the offloaded data are integrated into the centralized global map. Once replenished, the UAV resumes its exploration and mapping mission.

5.2 Software Architecture

As illustrated in Figure 5.3, the system is organized into a modular, multi-layer architecture to ensure clear separation of concerns between deployment, orchestration, and execution.

- **Platform Abstraction Layer :** Handles the containerized deployment via Docker to ensure strict dependency isolation across environments.
- **Execution & Orchestration Layers :** Manage the automated loading of the environment, procedural world generation, and the routing of different MARL training scenarios.
- **Simulation & RL Modules :** The core operational branches where Gazebo physics and PettingZoo environments execute simultaneously, synchronized by the step-locked mechanism.

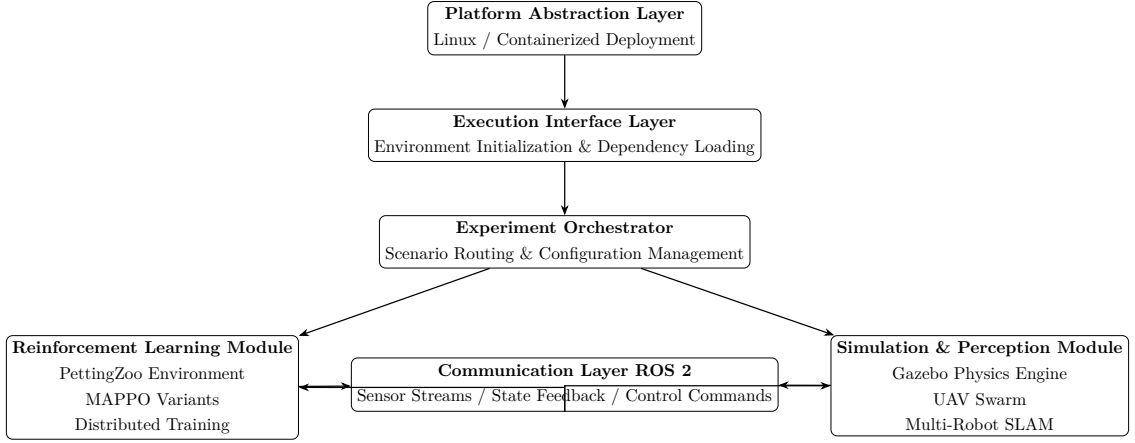


Figure 5.3: Modular multi-layer architecture of the proposed decentralized MARL framework. The system separates deployment, orchestration, reinforcement learning, and simulation modules, interconnected through a ROS 2 communication layer.

ROS 2 Middleware To enable communication between the physics engine and the learning algorithms, a `ros_gz_bridge` node ensures bidirectional exchange between Gazebo and ROS 2. The architecture follows a publisher–subscriber model where the learning environment subscribes to sensor data such as LiDAR, IMU and odometry, while the MARL policy publishes control commands received by the motor controllers. Topic mappings and message types are detailed in Table 5.1.

5.3 Training Pipeline

Table 5.1: Complete mapping of Gazebo and ROS 2 topics via the `ros_gz_bridge`.

Topic Type	Gazebo Topic	ROS 2 Topic	Message Type	Direction
Forward LiDAR	/model/{name}/lidar/points/points	/ {name}/velodyne_points	sensor_msgs/PointCloud2	Gz → ROS
Down LiDAR	/model/{name}/lidar_down/points/points	/ {name}/lidar_down/points	sensor_msgs/PointCloud2	Gz → ROS
Front Camera	/model/{name}/camera/image_raw	/ {name}/camera/image_raw	sensor_msgs/Image	Gz → ROS
Camera Info	/model/{name}/camera/camera_info	/ {name}/camera/camera_info	sensor_msgs/CameraInfo	Gz → ROS
GPS	/model/{name}/navsat	/ {name}/gps	sensor_msgs/NavSatFix	Gz → ROS
IMU	/model/{name}/imu	/ {name}/imu	sensor_msgs/Imu	Gz → ROS
Odometry	/model/{name}/odometry	/ {name}/odometry	nav_msgs/Odometry	Gz → ROS
Pose TF	/model/{name}/pose	/tf	tf2_msgs/TFMessage	Gz → ROS
Velocity Command	/model/{name}/cmd_vel	/ {name}/cmd_vel	geometry_msgs/Twist	ROS → Gz

The placeholder `{name}` corresponds to the specific agent identifier such as `uav_0` or `uav_1`.

5.3 Training Pipeline

We train our MARL architecture using **Ray RLlib**, a widely adopted library for distributed reinforcement learning. It provides support for **centralized training with decentralized execution**, scales efficiently across CPUs and GPUs, and allows integration of custom components such as our **Graph Attention Networks** and **Control Barrier Function** layers.

Because the problem involves multiple interacting agents, we use the **PettingZoo** framework rather than the single-agent **Gymnasium** API. We implemented a fully custom PettingZoo environment to model the UAV scenario, including **continuous 3D dynamics**, **147-dimensional observation vectors**, and reward functions capturing **energy consumption**, **collisions**, and **safety constraints**.

The environment follows the **Agent Environment Cycle** structure, which evaluates agents sequentially and reduces race conditions when drones operate in close proximity. For compatibility with RLlib’s parallel training, we wrap it using the `ParallelPettingZooEnv` interface so that actions can be processed simultaneously during learning.

At each step, the environment communicates with **ROS 2** by publishing velocity commands to `/cmd_vel`, advancing the **Gazebo** simulation, and retrieving updated odometry and LiDAR data before returning observations and rewards to the trainer.

5.3.1 Neural Network Architecture and Hyperparameters

The MAPPO framework follows a centralized training with decentralized execution paradigm, separating policy generation from value estimation. The model is implemented in PyTorch using a dual Actor–Critic architecture augmented with an explicit communication module.

To enable efficient multi-agent learning, we adopt a parameter sharing strategy in which a default swarm of **three UAVs** uses a single shared policy network. The data flow through the network is organized as follows:

- **Graph Attention Layer**

- *Input* : The local state of the agent and the states of neighboring agents within a 20-meter communication radius.
- *Output* : A context embedding capturing the most relevant relational information.

- **Actor Network**

- *Input* : A 147-dimensional observation vector composed of 14 kinematic features from odometry, 11 sparse features from the forward LiDAR, and 121 values from the flattened local occupancy grid, concatenated with the graph-based context embedding.
- *Output* : A 4-dimensional continuous action vector corresponding to the linear velocities v_x, v_y, v_z and yaw rate ω_z , normalized in the range $[-1, 1]$.

- **Centralized Critic Network**

- *Input* : The global state obtained by concatenating the observation vectors of all three UAVs, resulting in a 441-dimensional input.
- *Output* : A scalar value $V(s)$ representing the expected return, used during training for Generalized Advantage Estimation.

Both the Actor and the Critic are implemented as fully connected multi-layer perceptrons with two hidden layers of 256 units each. The main hyperparameters are summarized in Table 5.2.

Table 5.2: Core Hyperparameters for the MAPPO Actor-Critic Training Pipeline.

Hyperparameter	Value
Default Swarm Size	3 UAVs
Learning Rate	10^{-4}
Discount Factor γ	0.99
GAE Parameter λ	0.95
PPO Clipping Parameter	0.2
Entropy Coefficient	0.01
Training Batch Size	4000 samples
Actor and Critic Hidden Layers	[256, 256]

5.3.2 Role of Multi-Robot Graph SLAM within the Reinforcement Learning Loop

The integration of the **mrg_slam** framework constitutes the perception backbone of the system. In GPS-denied urban environments, accurate localization and mapping are prerequisites for stable policy learning. The distributed SLAM module contributes at three key stages of the control pipeline.

State Estimation for Policy Input Raw IMU and 3D LiDAR measurements are processed by the SLAM front-end to produce high-frequency odometry estimates. These estimates populate the 14 kinematic components of the observation vector, providing the policy with reliable information about position, velocity, and orientation.

Local Occupancy Representation LiDAR point clouds are registered into a local voxel grid, which is flattened into the 121-dimensional occupancy component of the observation space.

Global Mapping for Reward Computation To compute the exploration reward R_{inc} , the environment maintains a coverage grid. The downward-facing LiDAR updates this grid in real time. When agents enter communication range, their local SLAM backends exchange compact spatial keyframes. A decentralized pose graph optimization procedure aligns trajectories, corrects drift, and fuses observations into a globally consistent map. This shared representation ensures accurate reward attribution, **discourages redundant exploration**, and promotes coordinated coverage of unexplored regions.

5.4 Experimental Design and Monitoring

To evaluate the proposed framework, we establish a evaluation protocol and deploy specialized monitoring tools to track the learning dynamics in real-time.

Benchmark Scenarios

We define three experimental scenarios representing different constraint enforcement strategies.

1. **Scenario A - Baseline MAPPO** - This scenario relies exclusively on standard reward shaping to avoid collisions and manage energy. It offers the fastest convergence rate but provides the least reliable safety guarantees.
2. **Scenario B - Lagrangian MAPPO** - This approach incorporates soft constraints using adaptive Lagrange multipliers. The system tracks the average cost rate per episode and dynamically updates the penalty multiplier λ if the violation rate exceeds a strict 5% threshold.
3. **Scenario C - CBF MAPPO** - This scenario implements Control Barrier Functions as a hard safety filter directly on the action space, guaranteeing zero safety violations at the cost of slower policy convergence.

Performance Metrics and Formulas

The evaluation framework tracks several critical metrics to quantify the mapping performance and operational safety

- **Coverage Rate** : The primary mission objective, defined as the fraction of the environment successfully mapped. It is computed from the 2D grid as

$$Cov = \frac{N_{visited}}{N_{total}} \quad (5.1)$$

where $N_{visited}$ is the number of explored 0.5-meter cells and N_{total} is the total navigable area of the 200×200m arena.

- **Constraint Violation Frequency (C_{rate})** : The normalized rate of safety infractions per episode, used to measure the effectiveness of the Lagrangian and CBF methods

$$C_{rate} = \frac{\Sigma \text{Violations}}{N_{steps}} \quad (5.2)$$

5.4 Experimental Design and Monitoring

where tracked violations include physical collisions, No-Fly Zone intrusions, and altitude boundary breaches.

- **Success Rate and Battery Efficiency :** Operational performance is further measured through the final battery efficiency, representing the percentage of remaining energy upon mission completion or forced return.

Training Monitoring and Visualization

The training process is monitored using a dual visualization interface to ensure stability and analyze convergence behaviors. First, the **Ray Dashboard** provides real-time monitoring of the distributed system, displaying CPU and GPU utilization alongside worker logs. Second, all training logs are aggregated and streamed to **TensorBoard**. This allows for the historical analysis of training curves, including the actor and critic losses, the evolution of the entropy coefficient, and the adaptation of the Lagrangian multipliers. These tools are fundamental for detecting potential instabilities such as policy collapse or gradient explosions early in the training phase.

Summary

This chapter presented the complete technical realization of the proposed decentralized MARL system, from the Gazebo physics simulation and ROS 2 middleware integration to the detailed configuration of the Ray RLlib training pipeline and Multi-Robot Graph SLAM perception. The established experimental scenarios, alongside the defined performance and safety metrics, form the basis for evaluating the system effectiveness.

6

Results and Discussion

This chapter evaluates the performance of the proposed framework through extensive simulations. We analyze the effectiveness of the constrained approach, discuss the technical hurdles encountered during deployment, and suggest future research directions.

6.1 Experimental Results and Proposed Solutions

The evaluation focuses on the operational integrity of the GAT-based MAPPO architecture. The results indicate that the integration of decentralized perception and reinforcement learning is architecturally coherent despite the current limitations in policy convergence.



Figure 6.1: Procedurally generated urban environment in Gazebo Harmonic featuring randomized building geometries and No-Fly Zones

6.1.1 Validation and Execution Metrics

The primary success of the current implementation lies in the seamless coordination between the physics engine and the learning environment. Quantitative benchmarks obtained during execution trials on the local workstation provide evidence of system robustness

6.1 Experimental Results and Proposed Solutions

- **Simulation Throughput :** The Gazebo environment maintained a stable real-time factor of 0.8 with a control frequency of 20Hz for the MARL policy.
- **Middleware Latency :** The ROS 2 bridge effectively handled the sensor streams with an average transport latency of 15 milliseconds. This low overhead is critical for the Graph Attention Network to aggregate neighbor states without introducing significant lag in the decision loop.
- **Learning :** Although full convergence was not achieved, the training logs exhibit a consistent downward trend in policy loss during the first 100,000 training steps, suggesting that the reward structure provides a meaningful learning signal. This trend suggests that the reward signal successfully guides agents toward frontier zones while respecting battery constraints.

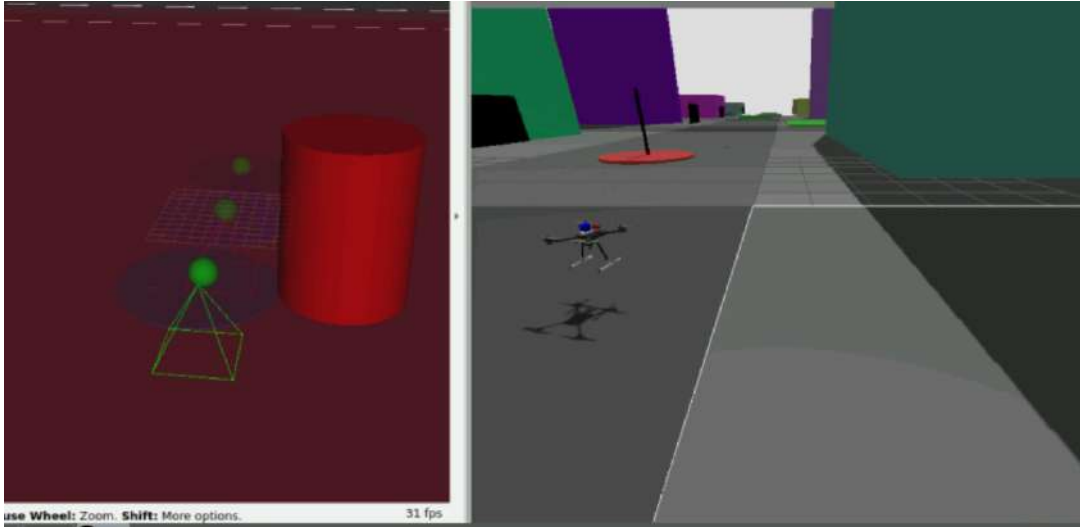


Figure 6.2: Real-time dual visualization proving the functional link between the physical Gazebo simulation on the right and the MARL perception state on the left.

6.1.2 Action Space Complexity and Deployment Strategy

The decision to focus on 2.5D navigation was a necessary response to the high-dimensional complexity of full 3D control.

- **Inference Benchmarks :** Fixing the altitude reduced the action space to three continuous variables per agent. This simplification allowed for a rigorous verification of the horizontal navigation logic and obstacle avoidance modules using the available hardware resources.

- **System** : These trials confirm that the entire software stack is stable and ready for deployment on high-performance clusters. The current project provides a foundational proof of concept that the **framework can handle decentralized SLAM and MARL concurrently**.

6.2 Implementation Challenges

The deployment phase identified the specific hardware and software barriers that prevent local MARL convergence in high-fidelity simulations.

Compute and Video RAM Saturation : Access to the **high-performance NVIDIA L40S cluster** was restricted to the initial month of the project. Subsequent training on an NVIDIA RTX 3050 workstation encountered severe memory bottlenecks. The concurrent execution of Gazebo and Ray Rllib reached 7.8 gigabytes of Video RAM usage out of the 8 gigabytes available. This saturation limited the training batch size to 4000 samples which increased gradient noise and hindered the optimization of the MAPPO objective.

Quadratic Program Infeasibility : Testing the CBF safety filter in dense urban canyons revealed the Leaky Corner problem. When multiple building constraints intersect the Quadratic Program solver occasionally fails to find a feasible solution. This mathematical bottleneck introduces non-deterministic latency in the control loop which disrupts the synchronization between the simulator and the RL agent.

SLAM and Grid Synchronization : Maintaining grid consistency at a 0.5-meter resolution across a 200-meter arena required significant memory management. Discrepancies between local voxel maps and the global map initially caused erratic reward signals. This was resolved by implementing a strict loop closure threshold within the decentralized SLAM pipeline to ensure map stability during the execution phase.

6.3 Discussion and Critical Analysis

The experimental outcomes indicate that building a functional decentralized MARL environment is as much an engineering task as an algorithmic one. The success of the hybrid architecture demonstrates that mission objectives like battery management and mapping can be modularized to reduce the learning burden on the stochastic policy.

The project results provide evidence that the proposed framework is architecturally coherent and suitable for large-scale training under adequate computational resources. The lack of a final converged policy on the local workstation does not invalidate the framework design but rather confirms the necessity for GPU-accelerated simulation environments. The tension between exploration depth and safety enforcement remains the central challenge and the 2.5D simplification served as a critical baseline for validating the system-level interactions.

6.4 Perspectives and Future Work

To bridge the gap between functional execution and industrial-scale autonomy we propose two primary technical evolutions focused on scalability and data efficiency.

Massive Parallelization and NVIDIA Isaac Lab - The current CPU-bound limitation of Gazebo will be resolved by migrating the environment to **NVIDIA Isaac Lab**. By leveraging GPU-accelerated physics through the PhysX engine we can simulate thousands of environments in parallel on HPC clusters. This transition is essential to explore the full 3D action space and reach policy convergence within reasonable timeframes.

Offline RL and Decision Transformers - To bypass the limitations of online simulation we propose a shift toward **Offline Reinforcement Learning**. By utilizing datasets of urban mapping missions we could train agents using **Decision Transformers** or **Conservative Q-Learning**. This approach would allow the swarm to learn from complex corner cases without the overhead of real-time physics updates while replacing the current MLPs with **Spatio-Temporal Transformers** for more efficient 3D data processing.

Summary

The results confirm that the GAT-MAPPO framework is architecturally sound and capable of coordinating a UAV swarm for autonomous mapping. While hardware constraints limited the project to execution validation and functional testing reaching 100,000 steps the successful SLAM integration provides a solid foundation for future large-scale training using Isaac Lab and Offline RL paradigms.

Conclusion

Decentralized multi-UAV systems are an important step for autonomous mapping in complex cities. This work shows that building these systems is not just an algorithmic challenge. It is a complex multidisciplinary research challenge that requires integrating perception, safety, communication, and learning. We model the task as a **Decentralized Partially Observable Markov Decision Process**. This creates a Multi-Agent Reinforcement Learning framework that works well with limited sensors and strict safety rules. A key contribution of this thesis is to **add Control Barrier Functions** as a formal safety layer inside the learning loop. We frame these safety rules as a Quadratic Program. This mathematically filters the neural network’s velocity commands to avoid obstacles and stay out of No-Fly Zones. This deterministic safety layer reduces the usual risks of reinforcement learning. It guarantees safe flights even before the policy fully converges. Due to hardware limits, the system did not reach full policy convergence. However, the simulation runs stably and shows good early learning results.

In the future, we will explore multimodal learning to combine IMU, LiDAR, and vision data. We also plan to test offline reinforcement learning, such as Decision Transformers, to train the system faster and improve generalization. Overall, this work proves that decentralized MARL is a safe, viable, and scalable foundation for drone swarms in critical environments.

References

- [1] JOSHUA ACHIAM. **Spinning Up in Deep Reinforcement Learning**. 2018. iv, 8
- [2] STEFANO V. ALBRECHT, FILIPPOS CHRISTIANOS, AND LUKAS SCHÄFER. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. iv, 10, 11, 13, 16, 20, 22
- [3] MOHAMAD A. HADY, SIYI HU, MAHARDHIKA PRATAMA, ZEHONG CAO, AND RYSZARD KOWALCZYK. **Multi-agent reinforcement learning for resources allocation optimization: a survey**. *Artificial Intelligence Review*, **58**, 2025. iv, 17, 18
- [4] PIERRE-YVES LAJOIE AND GIOVANNI BELTRAME. **Swarm-SLAM: Sparse Decentralized Collaborative Simultaneous Localization and Mapping Framework for Multi-Robot Systems**. *IEEE Robotics and Automation Letters*, **9**(1):475–482, 2024. iv, 22
- [5] CORA A. DIMMIG, GIUSEPPE SILANO, KIMBERLY MCGUIRE, CHIARA GABELLIERI, WOLFGANG HÖNIG, JOSEPH MOORE, AND MARIN KOBILAROV. **Survey of Simulators for Aerial Robots: An Overview and In-Depth Systematic Comparisons [Survey]**. *IEEE Robotics and Automation Magazine*, **32**(2):153–166, June 2025. iv, 25
- [6] D. KOLDASBAYEVA ET AL. **Challenges in data-driven geospatial modeling for environment monitoring: A scoping review**. *Nature Scientific Reports*, 2024. 1
- [7] LUBNA OBAID, KHALED HAMAD, RAMI AL-RUZOUQ, SALEH ABU DABOUS, KARIM ISMAIL, AND EMRAN ALOTAIBI. **State-of-the-art review of unmanned aerial vehicles (UAVs) and artificial intelligence (AI) for traffic and safety analyses: Recent progress, applications, challenges, and opportunities**. *Transportation Research Interdisciplinary Perspectives*, **33**:101591, 2025. 1

-
- [8] JINGTAO TANG, ZINING MAO, AND HANG MA. **Large-Scale Multirobot Coverage Path Planning on Grids With Path Deconfliction.** *IEEE Transactions on Robotics*, **41**:3348–3367, 2025. 2
 - [9] ZAID TAHIR, WALEED TAHIR, AND SAAD ALI LIAQAT. **State Space System Modelling of a Quad Copter UAV.** *arXiv preprint arXiv:1908.07401*, 2019. 4
 - [10] MOHAMMAD DEHGHANI TEZERJANI, MOHAMMAD KHOSHNAZAR, MOHAMMAD-HAMED TANGESTANIZADEH, ARMAN KIANI, AND QING YANG. **A Survey on Reinforcement Learning Applications in SLAM**, 2025. 5, 19
 - [11] Z. WU, Z. WANG, W. XIE, Z. LIN, Y. WU, AND Y. MO. **3D Voxel-based Collaborative Path Planning for UAVs in Urban Emergency Response.** *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, **XLVIII-G-2025**:1573–1579, 2025. 6, 15
 - [12] O RODENBERG, EDWARD VERBREE, AND SISI ZLATANOVA. **INDOOR A* PATHFINDING THROUGH AN OCTREE REPRESENTATION OF A POINT CLOUD.** **IV-2/W1**, 09 2016. 6
 - [13] RICHARD S. SUTTON AND ANDREW G. BARTO. *Reinforcement Learning: An Introduction.* The MIT Press, second edition, 2018. 7, 9
 - [14] R. J. WILLIAMS. **Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.** *Machine Learning*, **8**:229–256, 1992. 9
 - [15] J. SCHULMAN, S. LEVINE, P. ABBEEL, M. JORDAN, AND P. MORITZ. **Trust Region Policy Optimization.** In *ICML*, 2015. 10
 - [16] J. SCHULMAN, F. WOLSKI, P. DHARIWAL, A. RADFORD, AND O. KLIMOV. **Proximal Policy Optimization Algorithms.** In *arXiv preprint arXiv:1707.06347*, 2017. 10
 - [17] FENG WU, NICHOLAS R. JENNINGS, AND XIAOPING CHEN. **Sample-Based Policy Iteration for Constrained DEC-POMDPs.** In *Proceedings of the 20th European Conference on Artificial Intelligence (ECAI 2012)*, **242** of *Frontiers in Artificial Intelligence and Applications*, pages 858–863. IOS Press, 2012. 14
 - [18] AHMED SAEED, AHMED ABDELKADER, MOUHYEMEN KHAN, AZIN NEISHABOORI, KHALED A. HARRAS, AND AMR MOHAMED. **On Realistic Target Coverage by Autonomous Drones**, 2018. 15

REFERENCES

- [19] DENYS DATSKO, FRANTISEK NEKOVAR, ROBERT PENICKA, AND MARTIN SASKA. **Energy-Aware Multi-UAV Coverage Mission Planning With Optimal Speed of Flight.** *IEEE Robotics and Automation Letters*, **9**(3), 2024. 15
- [20] H. LI, P. XU, AND Y. CHEN. **Bio-Inspired Multi-UAV Coverage Path Planning Using Ant Colony and Particle Swarm Optimization.** *Applied Soft Computing*, **110**:107600, 2021. 16
- [21] R. ZHANG, D. WANG, AND J. HU. **A Hybrid ACO–PSO Algorithm for Cooperative Multi-UAV Path Planning in Dynamic Environments.** *Aerospace Science and Technology*, **138**:108358, 2023. 16
- [22] BOWEN BAKER, INGMAR KANITSCHIEDER, TODOR MARKOV, YI WU, GLENN POWELL, BOB MCGREW, AND IGOR MORDATCH. **Emergent Tool Use From Multi-Agent Autocurricula**, 2020. 16
- [23] YING WEN, YAODONG YANG, RUIQING WANG, FUXIANG WANG, YANG XU, AND CHONGJIE ZHAO. **A survey on multi-agent deep reinforcement learning.** *Neurocomputing*, **457**:153–171, 2021. 17
- [24] KAI ZHANG, JIAN WANG, ZHE LIU, AND HONG ZHANG. **A multi-agent deep deterministic policy gradient approach for multi-UAV 3D trajectory planning and collision avoidance.** In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5260–5266. IEEE, 2021. 19
- [25] WEI FU, YANG SONG, BOYANG LI, AND FEI GAO. **Multi-agent proximal policy optimization for large-scale UAV swarm formation and flight control.** *IEEE Transactions on Cognitive and Developmental Systems*, **15**(2):709–721, 2022. 19
- [26] SAINBAYAR SUKHBAATAR, ARTHUR SZLAM, AND ROB FERGUS. **Learning multi-agent communication with backpropagation.** In *Advances in neural information processing systems 29 (NIPS)*, 2016. 20
- [27] MENG QU, YUDA LIU, QIYUE CHEN, CHAO WANG, BO AN, AND YAIN-WHAR ZHENG. **MAGNNET: Multi-agent graph-attention networks for multi-agent reinforcement learning.** In *2020 International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 1018–1026, 2020. 20, 27

REFERENCES

- [28] RUI LI, SEN WU, SHUYING WU, AND XIAOPING CHEN. **G2ANet: Gated graph attention networks for multi-agent reinforcement learning**. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 759–767, 2020. 20
- [29] PIERRE-YVES LAJOIE, BENJAMIN RAMTOULA, FANG WU, AND GIOVANNI BELTRAME. **Towards Collaborative Simultaneous Localization and Mapping: A Survey of the Current Research Landscape**. *Field Robotics*, **2**:971–1000, May 2022. 20, 27
- [30] ANDREW Y NG, DAISHI HARADA, AND STUART RUSSELL. **Policy invariance under reward transformations: Theory and application to reward shaping**. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML '99)*, pages 278–287, 1999. 22
- [31] JOSHUA ACHIAM, DAVID HELD, AVIV TAMAR, AND PIETER ABBEEL. **Constrained policy optimization**. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 22–31. PMLR, 2017. 23
- [32] BOWEN LIU, ZIXU CEN, HONGHUA HE, YUHANG ZHOU, ZEQUING WANG, TENG LIU, AND HAO SU. **Safe multi-agent reinforcement learning for multi-robot-object-manipulation**. *IEEE Transactions on Robotics*, **39**(5):3987–4005, 2023. 24, 27
- [33] WEN-KAI LI, YA-CHU LI, WEI-YAO LEE, CHUN-AN LIN, KUAN-FU CHEN, AND KUAN-YI CHEN. **MAPCO: An effective multi-agent reinforcement learning algorithm with cost-constrained-based advantage function**. In *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1061–1068. IEEE, 2022. 24, 27
- [34] JASON J. CHOI, JASMINE JERRY ALOOR, JINGQI LI, MARIA G. MENDOZA, HAMSA BALAKRISHNAN, AND CLAIRE J. TOMLIN. **Resolving Conflicting Constraints in Multi-Agent Reinforcement Learning with Layered Safety**. *arXiv preprint arXiv:2505.02293*, 2025. 24, 27
- [35] H. M. SABBIR AHMAD, EHSAN SABOUNI, ALEXANDER WASILKOFF, PARAM BUDHRAJA, ZIJIAN GUO, SONGYUAN ZHANG, CHUCHU FAN, CHRISTOS CASSANDRAS, AND WENCHAO LI. **HMARL-CBF – Hierarchical Multi-Agent Re-**

inforcement Learning with Control Barrier Functions for Safety-Critical Autonomous Systems. *arXiv preprint arXiv:2507.14850*, 2025. 24, 27

- [36] CHAO YU, AKASH VELU, EUGENE VINITSKY, JIAXUAN GAO, YU WANG, ALEXANDRE M. BAYEN, AND YI WU. **The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.** In *Advances in Neural Information Processing Systems 35 (NeurIPS) Datasets and Benchmarks Track*, pages 24611–24624, 2022. 36

AI-Assistance Disclosure

This research proposal was written exclusively by **Prince Gedeon GUEDE**. I used AI-assisted tools, including Grammarly and DeepL, exclusively to enhance language clarity and grammar. **Gemini 3** was used exclusively to generate illustrative figures for the methodology section. All scientific ideas, models, analyses, and contributions presented in this document are original and solely attributable to the author.